

Learning Latent Protein Languages for Autoregressive Generation

Mahdi Pourmirzaei¹, Farzaneh Esmaili¹, Amir Ziashahabi², Mohammadreza Pourmirzaei³, and Dong Xu¹

¹University of Missouri, ²University of Southern California, ³Independent Researcher

Autoregressive transformers are the dominant generative recipe across most tokenized modalities, yet they remain comparatively weak for protein sequence and structure generation. We study the role of target representation in this gap: amino acid tokens encode residue identities without explicit contextual semantics, while backbone coordinates require a discrete representation in our framework. We introduce two learned latent protein languages. *Protein Latent Language* (PLL) maps sequences into a 4,096-state contextual alphabet built on a frozen ESM-2 encoder, retaining one token per residue. *Structure Latent Language* (SLL) adapts GCP-VQVAE Lite with auxiliary sequence and confidence supervision while preserving decoding to backbone coordinates. We separately pretrain autoregressive transformer language models on PLL and SLL tokens using a next-token prediction objective, yielding PLLM and SLLM. Under matched downstream sequence training, PLLM has a fitted compute-scaling exponent of 0.038 versus 0.020 for the amino acid autoregressive protein language model (PLM), a roughly $1.9\times$ steeper slope. SLLM has a fitted exponent of 0.049 on structure-token data. In unconditional sequence generation, PLLM substantially reduces the fraction of samples below a heuristic 1.5-bit residue-composition entropy threshold by 54% relative to the amino acid model across a sweep of sampling temperatures. At moderate sampling temperatures, PLLM better matches the sequence lengths and residue-composition entropy of the UniRef50 training data than PLM does. SLL improves performance on supervised protein structure tasks. For sequence-to-structure prediction, replacing the original GCP-VQVAE Lite tokenizer with SLL reduces best validation perplexity by 34% under matched training. In backbone generation, SLLM compares favorably with other generative models on diversity and novelty. We further use SLLM for sequence-to-structure prediction, where latent-token sampling for long proteins is approximately 1 000 times faster than MSA-based AlphaFold2 in our measurements, and we observe early signs that using the model’s own internal token confidence for inference-time scaling can lift prediction quality beyond a single decoded sample. Together, these results position learned latent protein languages as a promising direction for language modeling to explore, offering a practical substrate for bringing autoregressive transformer scaling and inference-time sampling to protein generation.

🔗 **Code Repository:** github.com/mahdip72/latent_protein_languages

🌐 **Project Page:** mahdip72.github.io/latent-protein-languages.github.io/

1. Introduction

Autoregressive transformer language models have become the central paradigm of frontier generative artificial intelligence (AI). Originally developed for text, the same decoder-only next-token recipe now operates as a unified multimodal interface across images [44], audio [55], video, and most recently embodied actions. These modalities are mapped to discrete tokens and generated by the same architecture [56, 35], as in vision-language-action systems for robotics [3]. The appeal of this paradigm is a single, scalable recipe that absorbs new modalities through tokenization rather than through architectural redesign.

Proteins are a natural target for this recipe, yet autoregressive modeling has not become the dominant

approach on either of their two modalities. On the sequence side, autoregressive protein language models exhibit power-law loss scaling with compute, while data repetition can limit gains from further training [5]. Models such as ProtGPT2 [10] and ProGen [34, 1] nevertheless remain less effective for protein design than non-autoregressive alternatives such as the discrete diffusion protein language model (DPLM) [48]. Their autoregressive sampling is also prone to drifting into pathological low-complexity, repetitive sequences that lack structural plausibility [57]. On the structure side, the gap is sharper because state-of-the-art de novo backbone generation is dominated by diffusion and flow-matching models [20, 50, 2], while autoregressive structure generation is essentially absent from the strongest baselines. The fact that the recipe powering frontier generative AI in other domains underperforms across both protein modalities is, in our view, the central problem.

We study how the choice of target representation affects autoregressive protein modeling. Amino acid tokens encode residue identity, while learned latent tokens can incorporate contextual information from a pretrained protein encoder. Raw three-dimensional coordinates require a discrete representation in this framework. Learned tokenizers in vision and audio motivate a similar approach for proteins: generate decodable latent tokens and recover the original observations. We therefore investigate learned target languages while retaining a standard autoregressive modeling recipe.

We address tokenization on both modalities and show that, once each side of the protein domain is mapped into a learned discrete latent language with high semantic density, autoregressive modeling becomes competitive with the non-autoregressive approaches on selected diversity and novelty metrics in our generation benchmarks. We introduce two such languages. *Protein Latent Language* (PLL) is a sequence-side discrete language built on top of ESM-2 [26] semantic representations using a vector-quantized representation autoencoder (VQ-RAE) tokenizer [9], and is decodable back to amino acid sequences. *Structure Latent Language* (SLL) is a structure-side discrete language designed to be reconstructable to backbone coordinates while remaining rich in structural and functional semantics. We then study PLL and SLL along the axes of scaling, de novo generation, and sequence-to-structure prediction. We refer to autoregressive models trained on PLL and SLL as protein latent language models (PLLMs) and structure latent language models (SLLMs), respectively. PLLM and SLLM are pretrained separately. For sequence-to-structure prediction, SLL generation is conditioned on E1 [21] sequence representations. The contributions of this study are summarized below.

- We introduce PLL, a contextual, residue-aligned target language derived from ESM-2 representations, and adapt GCP-VQVAE Lite into SLL through auxiliary supervision. Both languages remain decodable to amino acids or backbone coordinates.
- We show that PLLM has steeper fitted loss-versus-compute scaling and substantially less low-complexity drift than the amino acid model under matched downstream training.
- We apply SLLM to structure generation and sequence-to-structure prediction, combining confidence filtering adapted from DeepConf [11] with token-space consensus to select candidates before coordinate decoding.

2. Related Work

Autoregressive protein generation is most established for sequences. ProtGPT2 [10], RITA [17], ProGen2 [34], and ProGen3 [1] train decoder-only models over amino acid strings, while compute-optimal studies show that objective choice and data diversity shape scaling [5]. Structure-side autoregression is newer. Learning the Language of Protein Structure [14] tokenizes backbones with a vector-quantized autoencoder and trains a decoder-only transformer over structure codes. Structure Language Models [31] encode conformations with a discrete variational autoencoder and study conditional language models, including decoder-only

variants, for sequence-conditioned ensembles. Prot2Token [37] maps protein prediction targets, including three-dimensional structure, into next-token outputs from an autoregressive decoder. HelixProtX [4] uses a large multimodal-model interface for any-to-any sequence, structure, and description generation.

Non-autoregressive models set many current protein-generation baselines. DPLM [48] uses discrete diffusion for sequence generation, and DPLM-2 [49] extends diffusion to joint sequence-structure modeling with lookup-free quantized structure tokens. For structures, RFdiffusion [50], Chroma [20], FoldFlow [2], ProteinGenerator [27], and La-Proteina [15] use diffusion, flow matching, or programmable geometric generation to produce backbones or sequence-structure pairs with strong controllability and designability. These methods avoid a left-to-right sampling order and motivate our empirical comparison of whether autoregressive transformers become competitive when operating in learned protein latent languages.

3. Method

We start from the premise that the failure mode of autoregressive protein modeling is partly a representation problem. Instead of changing the next-token objective, we change the objects being predicted. We require a target language for autoregressive generative modeling to satisfy three properties. First, its vocabulary should be large enough to express fine-grained biological states, consistent with scaling-law evidence that larger language models benefit from larger vocabularies [42]. Second, its tokens should carry high-level semantics rather than only local symbols, following the role of compact latent tokenizers in visual reconstruction and generation [53]. Third, it should exhibit autoregressive friendliness, so the token sequence should be easy for a causal generator to model rather than optimized only for reconstruction fidelity [41]. For a protein sequence x and backbone structure y , we therefore learn discrete tokenizations that map each modality into such an ordered latent sequence. The sequence tokenizer defines PLL, with codes that can be decoded to amino acid identities. The structure tokenizer defines SLL, with codes that can be decoded to backbone coordinates.

With this interface fixed, we train decoder-only transformers on raw amino acid, PLL, and SLL streams using the same next-token factorization. The matched amino acid–PLL comparison evaluates the complete PLL target representation, including pretrained ESM-2 semantics and discretization. This comparison does not isolate the effects of ESM-2 semantics, vector quantization, vocabulary expansion, or learned tokenizer transformations. SLL is studied separately on structure data. We refer to models that generate PLL or SLL streams as autoregressive latent language models, while the raw amino acid model serves as a non-latent autoregressive reference. In the following, we describe how these latent languages are constructed and how autoregressive models are used to generate them.

3.1. PLL

PLL provides a contextual re-alphabetization of protein sequences. Its role is to replace residue symbols with discrete tokens that inherit the semantic organization of a pretrained protein encoder while remaining decodable to amino acid sequences. The vector-quantization bottleneck follows established VQ-VAE [45] principles, while the final decodable language uses a VQ-RAE-style [9] construction. Given a sequence $x = (x_1, \dots, x_L)$, a frozen ESM-2 150M encoder [26] produces contextual representations $\mathbf{h}_{1:L}$. We use this encoder as a compute-efficient semantic backbone, since the Protein Foundation Model Benchmark [13] reports that larger ESM-2 variants give only marginal gains over ESM-2 150M relative to their inference cost. A transformer latent encoder maps these vectors to $\mathbf{u}_{1:L}$, and a codebook $\mathbf{C} = \{\mathbf{c}_k\}_{k=1}^K$ assigns each residue

position to a discrete code by negative Euclidean distance,

$$\ell_{t,k} = -\|\mathbf{u}_t - \mathbf{c}_k\|_2, \quad z_t^{\text{PLL}} = \arg \max_{k \in \{1, \dots, K\}} \ell_{t,k}, \quad \mathbf{q}_t = \mathbf{c}_{z_t^{\text{PLL}}}. \quad (1)$$

The result is an ordered PLL token sequence with one discrete code per residue. In our experiments, $K = 4096$. Multiple PLL codes can decode to the same amino acid, allowing the target alphabet to distinguish contextual states while preserving sequence length and residue alignment. During Stage 1 training, code assignments are sampled from the same distance scores, while evaluation and decoding use the deterministic rule in Equation (1). Implementation details are given in Section A.1. The two-stage pipeline in Figure 1 makes the construction explicit. Stage 1 learns the quantized bottleneck through latent reconstruction, while Stage 2 freezes this bottleneck and trains amino acid recovery from the fixed PLL codes.

Stage 1 trains the tokenizer as a VQ-VAE-style latent reconstruction model. The ESM-2 encoder is fixed, while the latent encoder and latent decoder D_θ are optimized around the codebook to reconstruct $\mathbf{h}_{1:L}$ from the quantized vectors $\mathbf{q}_{1:L}$, producing $\hat{\mathbf{h}}_{1:L} = D_\theta(\mathbf{q}_{1:L})$. The reconstruction target is therefore the ESM-2 latent representation, not the amino acid sequence. The Stage 1 training loss combines a latent reconstruction loss and a vector-quantization loss,

$$\mathcal{L}_{\text{PLL}}^{(1)} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{vq}} = 10 \mathcal{L}_{\text{mse}} + 1 \mathcal{L}_{\text{cos}} + 0.5 \mathcal{L}_{\text{commit}}. \quad (2)$$

Section A.1 gives the detailed definitions of these terms in Equation (6). The reconstruction metrics used for monitoring and diagnostics are described in Section A.2.

Stage 2 turns the learned codes into the final VQ-RAE-style PLL. Starting from the selected Stage 1 checkpoint, we freeze the latent encoder and codebook, discard the Stage 1 decoder weights, and train a reinitialized transformer decoder with a linear 21-class residue-classification head on the fixed quantized vectors. The classifier covers the 20 standard amino acids plus the unknown residue class X. The Stage 2 training loss, $\mathcal{L}_{\text{PLL}}^{(2)}$, is token-level residue cross-entropy over the fixed PLL code sequence. Because the latent encoder and codebook are fixed, amino acid recovery depends only on information stored in the discrete code sequence. At generation time, an autoregressive latent language model produces PLL code IDs, the fixed codebook maps them back to quantized vectors, and the Stage 2 transformer decoder and linear classification head predict amino acid identities as $\arg \max_c p_\phi(c \mid z_{1:L}^{\text{PLL}}, t)$. ESM-2 is used to construct PLL targets from observed sequences. PLLM sampling and residue decoding run without ESM-2.

3.2. SLL

SLL is the structure-side latent language. It builds on GCP-VQVAE [38], a geometry-complete vector-quantized autoencoder tokenizer for protein backbones that maps structures to a 4096-token discrete codebook and decodes those codes back to backbone coordinates. The original GCP-VQVAE work introduces Lite and Large variants. We use GCP-VQVAE Lite as the baseline architecture for SLL, so the comparison isolates the effect of the tokenizer objective rather than increasing model scale. We keep the Lite backbone-tokenizer interface and use it as the structural counterpart to PLL. Given a backbone y , a GCPNet encoder produces residue-level geometric embeddings, a transformer encoder contextualizes them, and vector quantization assigns one SLL code per residue. A transformer decoder then maps the quantized representation to backbone coordinates through a six-dimensional rotation head. For the single-chain monomers studied here, the input’s absolute translation and orientation are omitted. The decoder reconstructs the backbone in its own coordinate frame, and reconstruction is evaluated up to a global rigid transformation.

We adapt the GCP-VQVAE Lite training objective to study the tradeoff between reconstruction fidelity, semantic utility, and autoregressive predictability. The training objective still includes backbone reconstruction

and vector-quantization terms, but adds three auxiliary targets, shown in Figure 2. Two pre-decoder heads predict amino acid identity and per-residue confidence measured by the predicted Local Distance Difference Test (pLDDT) score from the quantized stream, encouraging the codes themselves to retain sequence and confidence information. A post-decoder head predicts frozen ESM-2 representations, encouraging the reconstructed hidden states to remain aligned with protein-sequence semantics. These auxiliary heads are used during tokenizer training only. After training, autoregressive latent language models operate on SLL code IDs, and generated SLL sequences are decoded back to backbones with the fixed structure decoder.

We select the final SLL recipe using the Protein Structure Tokenization benchmark (PST), also known as StructTokenBench [54], which evaluates whether tokenizer embeddings support functional, physicochemical, and structural prediction tasks beyond coordinate reconstruction. We treat the auxiliary heads and reconstruction-loss choices as tokenizer-design choices, and select the final active recipe by supervised PST while keeping the GCP-VQVAE Lite architecture and training corpus fixed. This selection criterion matches the role of SLL in the rest of the paper. The tokenizer must remain reconstructable, but its primary purpose is to provide an autoregressive-friendly structure language whose tokens carry information useful for generation and sequence-to-structure prediction. Architecture and metric details are given in Sections A.3 and A.4.

3.3. Autoregressive Protein Modeling

After PLL and SLL are fixed, we model them alongside raw amino acid sequences as three standalone protein languages. Each language is tokenized residue-wise, and each token stream is pretrained separately with a decoder-only autoregressive transformer using cross-entropy next-token prediction. This follows the standard Transformer [47] and GPT-2 [40] recipe, while changing only the protein token space. We use PLLM and SLLM for the autoregressive latent language models trained on PLL and SLL streams, respectively. For amino acid sequences, the same architecture is a non-latent autoregressive reference. For unconditional generation, amino acid, PLL, and SLL streams use the same simple format with a beginning token, a content sequence, and an end token. For controlled de novo generation, we condition the same autoregressive model by prepending special tokens, such as a length token, before the generated content. The appendix gives the full token inventory and example streams in Section A.5.

3.3.1. Sequence-to-Structure Generation

We also adapt the pretrained SLLM for sequence-to-structure generation, as shown in Figure 3. The input amino acid sequence is encoded outside the causal token stream with Profluent-E1 600M [21]. We use E1 because it provides strong structure-aware sequence representations at moderate encoder scale, including superior unsupervised contact-map prediction relative to comparable publicly available protein encoders. The resulting residue-level embeddings are projected into the decoder hidden dimension and supplied as continuous context. We use the sequence-to-structure task token to condition the decoder for structure generation, with example token streams shown in Table 9. The decoder then generates SLL structure tokens autoregressively, and the fixed SLL decoder maps the generated token sequence back to backbone coordinates.

3.3.2. Confidence-Based Candidate Selection

To test whether sequence-to-structure prediction benefits from inference-time scaling, we sample multiple SLL token sequences for the same input and score each candidate before decoding it back to coordinates. In an offline selection procedure adapted from Deep Think with Confidence (DeepConf) [11], we treat raw full-vocabulary entropy from the SLLM as an internal confidence signal for latent structure generation. The central question is whether confidence measured in SLL space correlates with final structure quality and can

select better candidates without ground truth. Oracle best-of- k is used only as an evaluation ceiling. The practical selector uses confidence summaries and token-space consensus over sampled SLL sequences, as described in Section A.6.

4. Experiments

We organize the experiments around the two layers of the approach. First, we evaluate whether the learned tokenizers define useful discrete protein languages. PLL should preserve sequence information through latent reconstruction and amino acid recovery, while SLL should preserve structure and improve the semantic utility of GCP-VQVAE Lite through auxiliary objectives. Second, after the tokenizers are fixed, we train an amino acid language model, PLLM, and SLLM and test whether the latent token spaces improve scaling and generation.

All sequence-side experiments use UniRef50 as the source corpus. SLL tokenizer experiments use AFDB representative structures, while SLLM generation uses the larger $\text{AFDB} \cap \text{UniRef50}$ structure-token corpus introduced by GCP-VQVAE [38]. Further data-source details are given in Section B. All experiments are implemented in PyTorch 2.8 [36]¹ and run on four nodes, each with eight NVIDIA A100 80 GB GPUs. Unless stated otherwise, training uses FlashAttention-2 (FA2) [7], mixed precision [33], AdamW [29], and cosine learning-rate decay [28].

4.1. Latent Tokenizers

4.1.1. PLL Results

The PLL tokenizer is evaluated at two points that matter for the rest of the paper. Stage 1 evaluates reconstruction fidelity in the frozen ESM-2 representation space. The final Stage 1 checkpoint reaches $\Delta\text{PPL}_{\%} = 0.47$, with median normalized mean squared error (NMSE) of 0.044, reconstruction Kullback-Leibler (KL) divergence of 0.003, cosine similarity of 0.970, and full code activation on the validation split. The $\Delta\text{PPL}_{\%}$ metric is the primary monitoring metric because it measures how much the reconstructed latent distribution changes relative to the original ESM-2 latent distribution. The latent reconstruction delta perplexity of 0.47% and cosine similarity of 0.970 indicate close reconstruction of the frozen ESM-2 representations on the validation split. Given the contextual protein information learned by ESM-2, these results are consistent with retention of that information in the discrete PLL representation.

Stage 2 then freezes the Stage 1 encoder and codebook and asks whether the resulting discrete PLL codes are sufficient to recover amino acid identities. The fixed codes achieve 99.81% micro token-level residue recovery on the UniRef50 validation split. All standard amino acids have class-wise recovery above 99.17%. Together, these results support using PLL as a decodable sequence-side latent language for the autoregressive experiments that follow.

4.1.2. SLL Results

The SLL experiments keep the GCP-VQVAE Lite architecture and training corpus fixed, and change the tokenizer objective. We refer to the selected SLL tokenizer as GCP-VQVAE Lite 2. The ablation study trains each candidate recipe on 200,000 samples from the main SLL tokenizer source corpus and uses supervised PST as the primary selection target because the new auxiliary heads are intended to improve the semantic

¹The transformer and vector-quantization layers build on [x-transformers](#) and [vector-quantize-pytorch](#).

utility of the structure codes. The combined recipe gives the strongest average supervised PST performance among the comparable ablations, improving over the base tokenizer by 24.2% in functional-site AUROC and 35.0% in physicochemical Spearman (Table 1). The full set of supervised ablation averages is given in Section C.2.1.

After training the selected recipe on the full SLL tokenizer setup, GCP-VQVAE Lite 2 improves supervised PST averages over the original Lite and Large tokenizers from the original work (Table 2). The detailed task-level table is given in Section C.2.2. Unsupervised PST scores remain high but are treated as secondary and reported in Section C.2.4.

Table 1: SLL supervised PST ablation averages.

Configuration	Functional AUROC (%) $\uparrow$	Physicochemical Spearman (%) $\uparrow$
Base	58.26	23.93
MSE + distance	61.28	30.32
pLDDT pre-dec.	62.65	28.95
Inverse folding 0.1	67.15	23.66
Combined	72.37	32.30

Table 2: Average supervised PST comparison with baseline tokenizers. Lite 2 denotes SLL, and parentheses give the four-layer MLP and transformer probe results for GCP-VQVAE variants [38]. Gray columns provide external context and are not used for bold and underline ranking.

Metric	FoldSeek [46]	ProTokens [24]	VanillaVQ [54]	AIDO [58]	ESM3 [16]	AminoAseed [54]	GCP-VQVAE Large [38]	GCP-VQVAE Lite [38]	GCP-VQVAE Lite 2
Functional AUROC (%) $\uparrow$	51.90	65.37	68.30	69.38	69.24	72.43	61.19 (68.10, 73.57)	61.84 (68.30, 74.93)	71.32 (71.65, 78.08)
Physicochemical Spearman (%) $\uparrow$	7.80	9.84	33.49	21.81	37.35	38.08	16.61 (17.89, 21.37)	20.84 (22.73, 23.89)	<u>29.63</u> (29.71 , 27.52)

SLL remains competitive with state-of-the-art reconstructive tokenizers. Its average TM-score is 2.4% below the original GCP-VQVAE Lite tokenizer and close to AIDO (0.9456 versus 0.9470), while its average RMSD is lower (1.4722 versus 1.8189 Å) (Table 3). SLL also achieves higher functional-site AUROC and physicochemical Spearman than AIDO in the reported supervised PST comparison. It retains the GCP-VQVAE Lite backbone, for which the original benchmark reported substantially lower inference latency than AIDO [38]. The full split-level reconstruction table is given in Section C.2.3.

Table 3: Average reconstruction quality over CASP14, CASP15, CASP16, CAMEO-2024, and the zero-shot set from the original benchmark [38], where Lite 2 is our SLL tokenizer. The grayed GCP-VQVAE Large column is not used for bold and underline ranking.

Metric	GCP-VQVAE Lite 2	GCP-VQVAE Large [38]	GCP-VQVAE Lite [38]	AIDO [58]	ESM-3 VQVAE [16]	DPLM-2 VQVAE [49]	FoldToken 4 [12]	Gaujac et al. [14]	AminoAseed [54]
Average TM-score $\uparrow$	0.9456	0.9844	0.9687	<u>0.9470</u>	0.9291	0.8311	0.8788	0.8394	0.4000
Average RMSD $\downarrow$	<u>1.4722</u>	0.6172	0.9364	1.8189	2.8905	7.3650	3.0106	6.0002	18.2178

Finally, we ask whether the improved SLL tokenizer is also a better target language for autoregressive sequence-to-structure prediction. In a matched Prot2Token [37]-style setup, changing only the target tokenizer reduces best validation perplexity by 34% and reaches a comparable training-loss regime 49% faster when progress is measured by training epoch (Figure 4). This suggests that SLL improves not only representation quality, but also autoregressive compatibility. Full setup details are given in Section C.2.5.

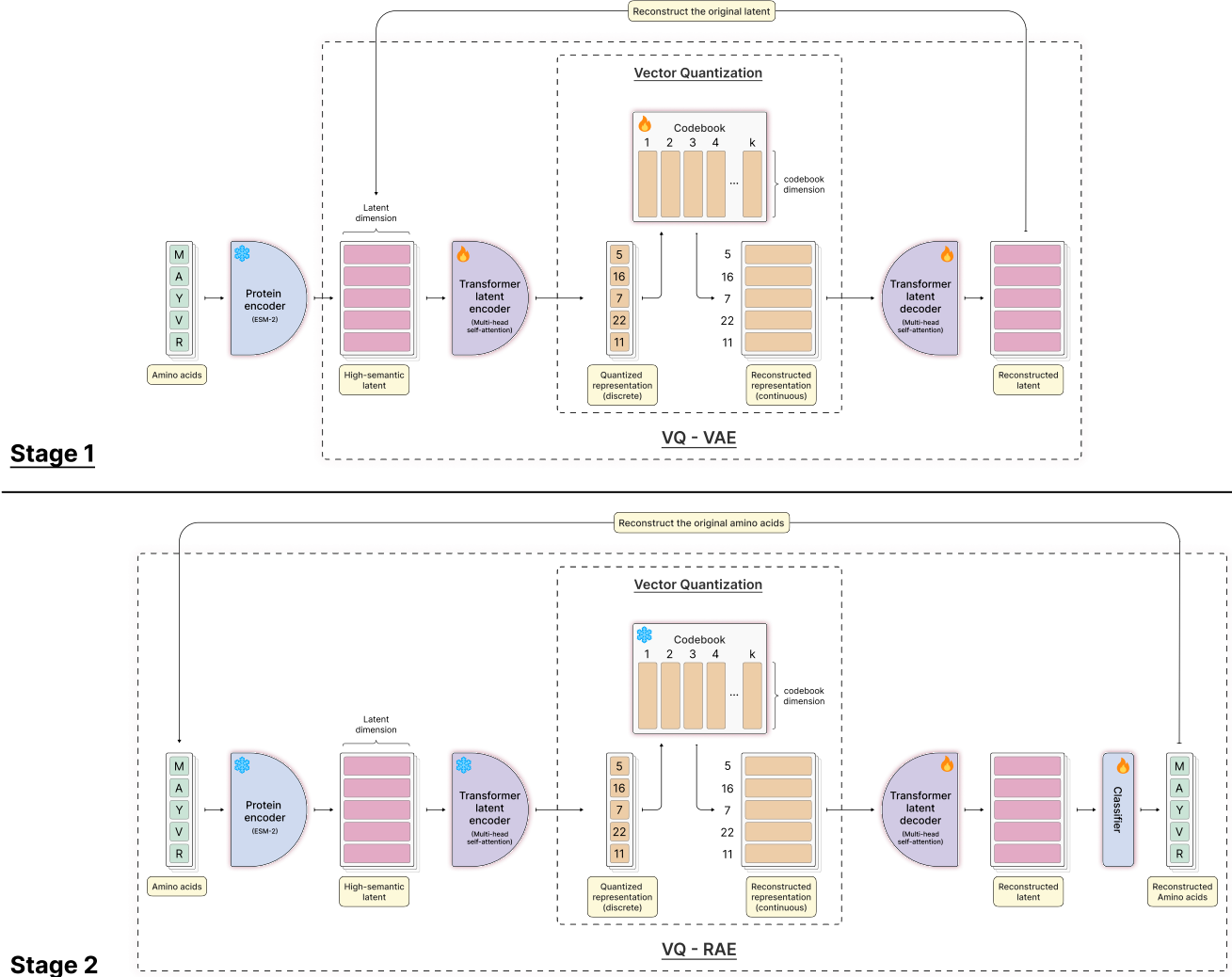


Figure 1: PLL construction. Stage 1 trains a vector-quantized variational autoencoder (VQ-VAE)-style latent reconstructor for frozen ESM-2 representations. Stage 2 finalizes PLL as a VQ-RAE by freezing the latent encoder and codebook and training a reinitialized transformer decoder with a linear residue-classification head.

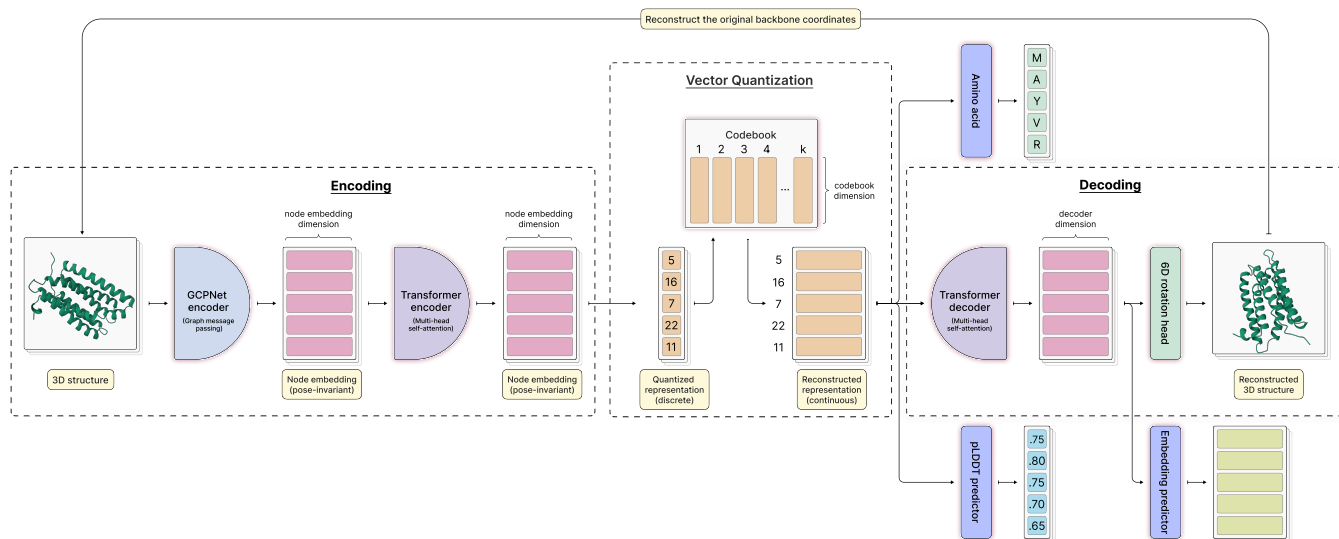


Figure 2: SLL construction. A GCPNet and transformer encoder convert backbone geometry into discrete structure codes. The decoder reconstructs backbone coordinates, while auxiliary heads train the codes to retain residue identity, confidence, and sequence-representation information.

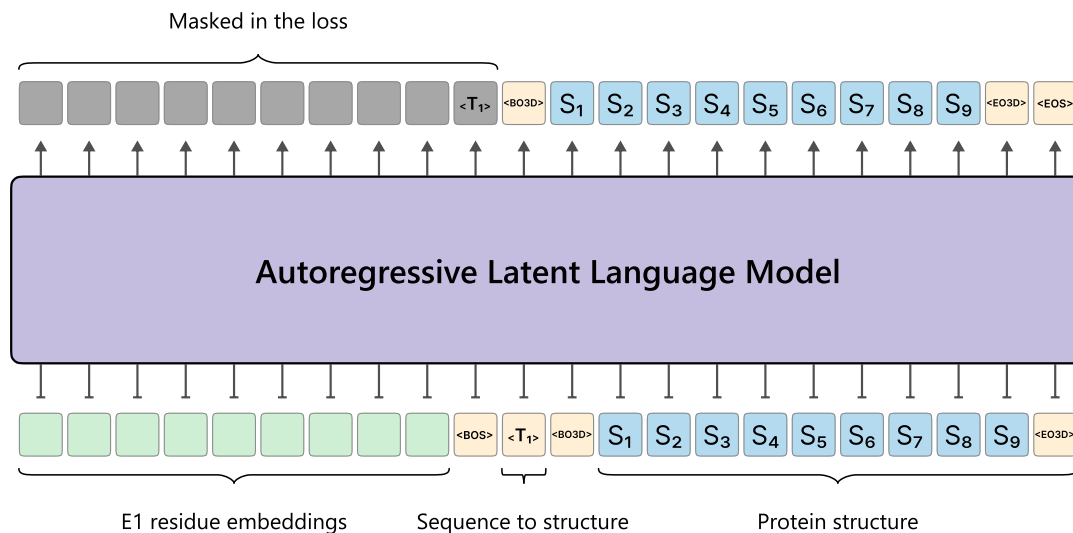


Figure 3: Sequence-to-structure generation via SLLM. E1 encodes the input sequence. The decoder predicts SLL tokens, which the SLL decoder maps to coordinates. Green, yellow, and blue denote protein embeddings, special, boundary, and condition tokens, and SLL tokens.

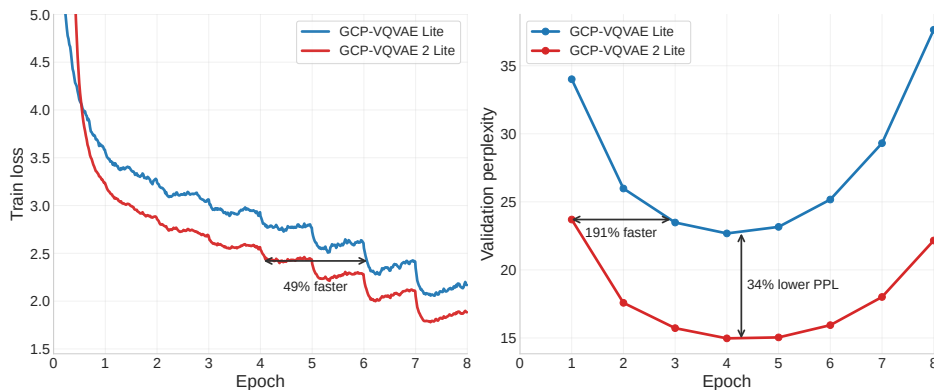


Figure 4: Matched sequence-to-structure tokenizer comparison. Left: train loss. Right: validation perplexity, where Lite 2 lowers the best value by 34%. The learning-speed annotations compare training progress by epoch.

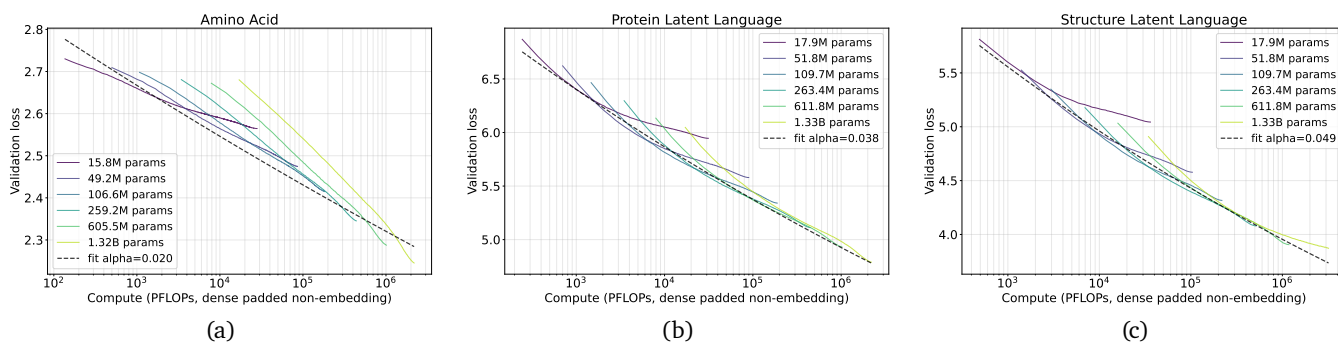


Figure 5: Validation loss versus estimated downstream autoregressive training compute. PLLM has a steeper fitted slope than the amino acid model under matched sequence training. SLLM is shown separately with structure-token data and different training exposure.

4.2. Autoregressive Protein Language Models

4.2.1. Do Latent Languages Scale Better?

We compare validation cross-entropy against estimated downstream autoregressive training compute for decoder-only transformers sharing the same architecture grid and next-token objective. The amino acid model and PLLM use the same UniRef50 sequences, token budget, and optimization settings. Their fitted exponents are $\alpha = 0.020$ and $\alpha = 0.038$, respectively (Figure 5). PLLM therefore has a roughly $1.9\times$ steeper fitted slope under matched downstream sequence training. SLLM has a fitted exponent of $\alpha = 0.049$ on structure-token data, with different training exposure. We compare fitted slopes rather than treating absolute cross-entropy across vocabularies as a common quality scale. The SLLM exponent is also numerically close to the exponent reported for text language models against minimum training compute adjusted for batch efficiency [23], although this comparison is qualitative because the corpora, tokenizers, compute accounting, and frontier construction differ. The fitting protocol, architecture grid, and training settings are given in Section A.7.

4.2.2. PLL De Novo Generation

We first compare unconditional generation from the 1.3B amino acid model and the 1.3B PLLM in the same decoded amino acid space. Direct amino acid generation shows substantial drift, defined by the entropy heuristic in Section A.8, visible as low sequence entropy from repeated residue patterns. This is most severe at low temperatures and often coincides with weak stopping behavior, where samples continue until the maximum context window instead of ending naturally with a $\langle\text{EOS}\rangle$ token. Across the 12-temperature unconditional sweep, 5 154/12 000 amino acid samples (43.0%) and 2 383/12 000 PLLM samples (19.9%) have decoded residue-composition entropy below 1.5 bits. PLLM therefore substantially reduces this diagnostic failure mode, although low-complexity samples remain. These counts describe unfiltered unconditional generations, separately from the length-conditioned and filtered folding evaluations below. As temperature increases, decoded PLLM samples move toward the UniRef50 source distribution, with $T = 0.6$ to $T = 1.0$ giving the strongest joint match between entropy and length.

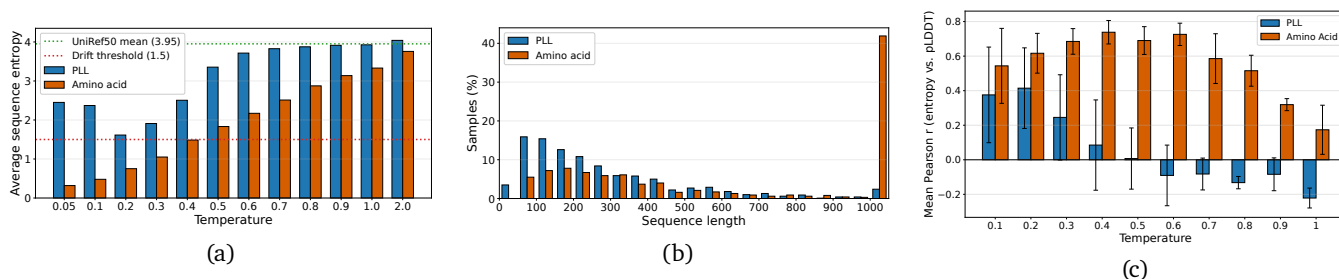


Figure 6: Amino acid versus PLLM sequence-generation diagnostics. Left: mean decoded residue-composition entropy across unconditional sampling temperatures. Middle: unconditional length distribution at $T = 1.0$. Right: length-conditioned entropy-pLDDT correlation after entropy-filtered resampling.

For length-conditioned generation, we continue fine-tuning the largest amino acid model and PLLM from their eight-epoch checkpoints. The length prefix is applied to 50% of continuation samples, so each checkpoint retains unconditional sampling while learning to generate at requested lengths; details are in Section C.3.2. Length conditioning gives both models strong target-length control, but amino acid composition entropy remains lower. This supports the conclusion that stopping and length behavior alone do not explain the complexity difference. Entropy-pLDDT correlations are reported separately for the entropy-filtered

Table 4: Length-conditioned sequence ProteinBench [52]. Native, ProGen2, DPLM, and ESM3 are published reference values. PLM is our amino acid model. Our PLM and PLLM use entropy-filtered samples evaluated with ESMFold.

Model	Quality	Diversity		Novelty
	pLDDT $\uparrow$	Pair. TM $\downarrow$	Clust. $\uparrow$	Max TM $\downarrow$
Native	63.70	0.5150	0.7900	–
ProGen2 [†] [34]	<u>65.26</u>	0.3900	0.9325	0.6675
DPLM [48]	89.93	0.5100	0.7450	0.8525
ESM3 [‡] [16]	53.24	0.3550	0.9750	0.6925
PLLM	59.86	<u>0.2567</u>	<u>0.9700</u>	<u>0.6368</u>
PLM [†]	<u>65.26</u>	0.2413	0.9250	0.5859

Unweighted means over lengths 100, 200, 300, and 500. Native, ProGen2, DPLM, and ESM3 values are taken from ProteinBench [52]. Our PLM and PLLM use $T = 0.5$, with 50 retained entropy-filtered samples per length evaluated using ESMFold. Length-specific results are reported in Table 12. [†] PLM uses additional sampling when needed to fill the entropy-filtered evaluation set. [‡] ProGen2 and ESM3 use roughly $15\times$ and $48\times$ more protein samples than UniRef50.

Table 5: Structure-design results on ProteinBench [52], averaged over target lengths 50, 100, 300, and 500.

Model	Quality		Diversity	Novelty	
	scTM $\uparrow$	scRMSD $\downarrow$	TM similarity $\downarrow$	Clust. $\uparrow$	Max TM $\downarrow$
Native*	0.9525	0.8250	0.2900	0.7500	–
Genie* [25]	0.4450	13.2375	0.2725	0.9650	0.4200
RFdiffusion* [50]	0.9200	1.8900	0.4200	0.6325	0.6675
Protpardelle [32]	<u>0.9289</u>	<u>1.5502</u>	0.7477	0.2175	0.7966
ProteinGenerator [27]	0.7524	2.5888	0.6554	0.4175	0.5806
PLAID [30]	0.7132	2.8963	0.5903	0.6525	0.6698
La-Proteina [15]	0.9677	0.7871	0.6231	0.6550	0.7369
Pallatom [39]	0.7991	2.3077	<u>0.5277</u>	<u>0.7375</u>	0.6296
FoldFlow-2 [19]	0.8857	1.6703	0.5770	0.6700	0.7684
DPLM-2 [†] [49]	0.8835	1.6511	0.7258	0.5375	0.9016
SLLM	0.5390	3.3974	0.4558	0.9175	0.5536

Bold and underline mark the best and second-best unstarred results. Diversity and novelty rankings require model-level mean scTM > 0.5 . * Results taken from ProteinBench. [†] DPLM-2 jointly models amino acid sequences and structure tokens. We evaluate its 650M model in unconditional backbone-generation mode.

length-conditioned samples.

In the entropy-filtered $T = 0.5$ comparison in Table 4, the amino acid model has higher mean pLDDT than PLLM (65.26 versus 59.86), lower pairwise TM, and lower Max TM. PLLM has the higher cluster ratio (0.9700 versus 0.9250). Its reduced unconditional drift and these filtered folding scores describe different sample populations.

Table 12 reports the corresponding values at each target length. Additional unconditional and length-conditioned generation analyses are provided in Sections C.3.1 and C.3.2.

4.2.3. SLL De Novo Generation

Unlike PLL, SLL does not have a raw-token counterpart that decodes to the same structure space. We therefore evaluate length-conditioned SLLM generation directly. Starting from the largest pretrained SLLM, we continue training for two epochs with length prefixes on 50% of samples, matching the conditioning interface in Section A.5. The generated SLL streams are decoded to backbone coordinates and evaluated with the structure-design ProteinBench [52] protocol. La-Proteina [15] achieves the highest mean scTM and lowest mean scRMSD (Table 5). SLLM generates diverse and novel backbones, with the highest cluster ratio and lowest Max TM among generators with mean scTM > 0.5 . Its self-consistency quality remains below that of specialized backbone generators. These results compare generation performance across pretrained systems whose training data, model sizes, and training modalities are not matched. Length-specific results are reported in Tables 13 and 14.

4.2.4. Sequence-to-Structure Prediction

We fine-tune the pretrained length-conditioned SLLM for sequence-to-structure prediction by connecting the E1 sequence encoder shown in Figure 3. The evaluated checkpoint sees 4.32B structure-token targets sampled from the same AF2-generated source used for SLL pretraining. At inference time, we sample multiple SLL structure-token trajectories per input sequence and use internal model confidence before coordinate decoding as an inference-time scaling signal.

We use a tuned DeepConf-style selector to test whether internal SLL-token confidence can identify better candidates before coordinate decoding. The resulting CAMEO-2024 curves are shown in Figure 7. Full selector settings, oracle comparisons, and additional metrics are provided in Section C.3.4.

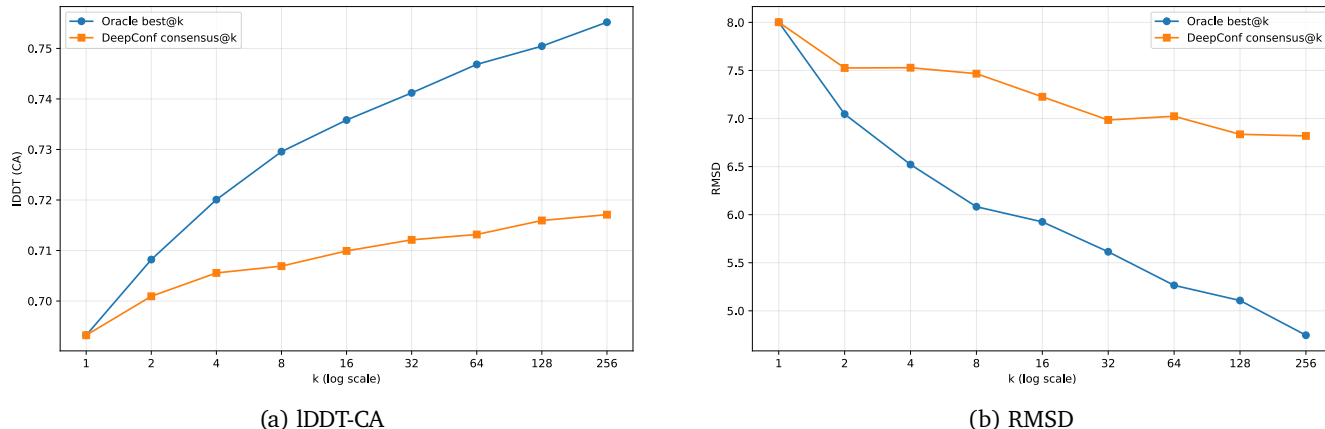


Figure 7: Sequence-to-structure inference-time scaling on the held-out CAMEO-2024 set as a function of the number of sampled SLL trajectories per input sequence (k). Oracle best@ k is a ceiling that picks the best candidate after coordinate decoding, while DeepConf consensus@ k uses only internal SLL-token confidence and token-space consensus to pick a single candidate before any coordinate decoding. (a) IDDT-CA $\uparrow$. (b) RMSD $\downarrow$. Both metrics improve overall as the sampling budget increases, and DeepConf recovers a meaningful fraction of the oracle gain. More details are given in Section C.3.4.

5. Discussion

The results support learned target representations as a useful way to improve autoregressive protein modeling. In the matched sequence comparison, PLLM shows steeper fitted compute scaling and substantially less low-complexity drift than amino acid modeling. SLL provides a complementary structure-side study, including improved target predictability in the controlled tokenizer comparison and a separate evaluation of backbone generation.

The backbone-generation results show a tradeoff between self-consistency quality, cluster diversity, and novelty relative to PDB references. La-Proteina leads the self-consistency quality metrics. SLLM’s strengths are diversity and novelty, while self-consistency quality remains a limitation.

PLL and SLL play different but complementary roles in this picture. PLLM addresses a major drawback of amino acid language models that common benchmarks can obscure. Direct residue-level generation often drifts into repetitive low-entropy patterns and weak $\langle \text{EOS} \rangle$ behavior, even when post-filtered samples can still score well on downstream folding diagnostics. Moving generation into PLL substantially reduces low-complexity drift in the unconditional sweep and better matches UniRef50 residue-composition entropy and length statistics at moderate temperatures. In the entropy-filtered length-conditioned comparison, the amino acid model has higher pLDDT, while PLLM has a higher cluster ratio. SLL shows that a structure tokenizer should not be judged only as a coordinate codec. By adding sequence, confidence, and representation supervision to the GCP-VQVAE Lite interface, SLL gives a small amount of reconstruction quality back in exchange for substantially stronger supervised PST performance and better autoregressive compatibility. Together, PLL and SLL frame latent protein languages as generative representations rather than compression artifacts.

The sequence-to-structure experiments highlight another consequence of this framing. In our measurements, SLLM generates structure-token candidates for long proteins approximately 1 000 times faster than MSA-based AlphaFold2 [22], making candidate sampling and selection practical. The oracle best@k curves show that better structures are present in the sampled SLL candidate pool as the number of samples increases. The DeepConf-style selector recovers part of this gain using only internal latent-token confidence and token-space consensus, making confidence before coordinate decoding a useful signal for protein structure generation.

Several limitations remain. The current models are not the strongest methods on quality-centered metrics, especially for de-novo backbone generation. While we apply SLLM in a cross-modal setting for sequence-to-structure prediction, each of our pretrained autoregressive transformers operates on a single token stream, and we do not evaluate a unified multimodal pretraining setup that jointly models sequence and structure tokens for generation. Joint PLL–SLL co-generation and compatibility between the independently learned languages on joint tasks remain untested. Future work should address both directions.

PLL is designed to represent the contextual protein information learned by ESM-2 as a discrete language that can be decoded to amino acid sequences. Our sequence comparison evaluates this complete target-language design. We did not compare it with continuous-target generation or similarly resourced ESM-2 fine-tuning and distillation approaches. Such comparisons would assess the generation performance of the complete PLL approach relative to alternative uses of pretrained protein representations.

Extending SLL to multichain complexes would require modeling relative chain poses, which we have not evaluated here.

6. Conclusion

We presented two learned latent protein languages, PLL on the sequence side and SLL on the structure side, and used them as drop-in target vocabularies for standard decoder-only transformers. Under matched downstream sequence training, PLLM shows steeper fitted compute-scaling trends and substantially less low-complexity drift than amino acid modeling. SLL improves autoregressive target predictability in the controlled tokenizer comparison and supports structure generation and sequence-to-structure prediction. We see this as early evidence that learned latent protein languages are a promising direction for autoregressive recipes, and we hope it motivates further work on using this approach for protein sequence and structure generation.

Acknowledgments

We gratefully acknowledge NVIDIA and the National Artificial Intelligence Research Resource (NAIRR) Pilot program for providing the computational resources that made this work possible, under project NAIRR240246.

References

- [1] Aadyot Bhatnagar et al. Scaling unlocks broader generation and deeper functional understanding of proteins. *bioRxiv*, 2025. doi: 10.1101/2025.04.15.649055. ProGen3 family of generative protein language models.
- [2] Avishek Joey Bose, Tara Akhound-Sadegh, Guillaume Huguet, Kilian Fatras, Jarrod Rector-Brooks, Cheng-Hao Liu, Andrei Cristian Nica, Maksym Korablyov, Michael Bronstein, and Alexander Tong.

- SE(3)-stochastic flow matching for protein backbone generation. In *International Conference on Learning Representations (ICLR)*, 2024.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
 - [4] Zhiyuan Chen, Tianhao Chen, Chenggang Xie, Yang Xue, Xiaonan Zhang, Jingbo Zhou, and Xiaomin Fang. Unifying sequences, structures, and descriptions for any-to-any protein generation with the large multimodal model HelixProtX. *arXiv preprint arXiv:2407.09274*, 2024.
 - [5] Xingyi Cheng, Bo Chen, Pan Li, Jing Gong, Jie Tang, and Le Song. Training compute-optimal protein language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
 - [6] Andrea Coletta, John W. Pinney, David Y. W. Solís, Joseph Marsh, Steve R. Pettifer, and Teresa K. Attwood. Low-complexity regions within protein sequences have position-dependent roles. *BMC Systems Biology*, 4:43, 2010. doi: 10.1186/1752-0509-4-43.
 - [7] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
 - [8] Justas Dauparas, Ivan Anishchenko, Nathaniel Bennett, Hua Bai, Robert J. Ragotte, Lukas F. Milles, Basile I. M. Wicky, Alexis Courbet, Rob J. de Haas, Neville Bethel, Philip J. Y. Leung, Timothy F. Huddy, Samuel Pellock, Doug Tischer, Frederick Chan, Brian Koepnick, Hanne Nguyen, Alex Kang, Banumathi Sankaran, Asim K. Bera, Neil P. King, and David Baker. Robust deep learning-based protein sequence design using ProteinMPNN. *Science*, 378(6615):49–56, 2022. doi: 10.1126/science.add2187.
 - [9] Sinan Du, Jiahao Guo, Bo Li, Shuhao Cui, Zhengzhuo Xu, Yifu Luo, Yongxian Wei, Kun Gai, Xinggang Wang, Kai Wu, and Chun Yuan. VQRAE: Representation quantization autoencoders for multimodal understanding, generation and reconstruction. *arXiv preprint arXiv:2511.23386*, 2025.
 - [10] Noelia Ferruz, Steffen Schmidt, and Birte Höcker. ProtGPT2 is a deep unsupervised language model for protein design. *Nature Communications*, 13(1):4348, 2022. doi: 10.1038/s41467-022-32007-7.
 - [11] Yichao Fu, Xuewei Wang, Hao Zhang, Yuandong Tian, and Jiawei Zhao. Deep think with confidence. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=8LqHs0KIM7>.
 - [12] Zhangyang Gao, Cheng Tan, Yijie Zhang, Xingran Chen, Lirong Wang, and Stan Z Li. FoldToken: Learning protein language via vector quantization and beyond. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. doi: 10.1609/aaai.v39i1.31998.
 - [13] Zhangyang Gao, Hao Wang, Cheng Tan, Chenrui Xu, Mengdi Liu, Bozhen Hu, Linlin Chao, Xiaoming Zhang, and Stan Z. Li. PFMBench: Protein foundation model benchmark. *arXiv preprint arXiv:2506.14796*, 2025.
 - [14] Benoit Gaujac, Jérémie Donà, Liviu Copoiu, Timothy Atkinson, Thomas Pierrot, and Thomas D Barrett. Learning the language of protein structure. *arXiv preprint arXiv:2405.15840*, 2024.
 - [15] Tomas Geffner, Kieran Didi, Zhonglin Cao, Danny Reidenbach, Zuobai Zhang, Christian Dallago, Emine Kucukbenli, Karsten Kreis, and Arash Vahdat. La-proteina: Atomistic protein generation via partially latent flow matching. In *International Conference on Learning Representations (ICLR)*, 2026.

- [16] Tomas Hayes, Roshan Rao, Halil Akin, Nicholas J Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q Tran, Jonathan Deaton, Marius Wiggert, et al. Simulating 500 million years of evolution with a language model. *Science*, 2025. doi: 10.1126/science.ads0018.
- [17] Daniel Hesslow, Niccoló Zanichelli, Pascal Notin, Iacopo Poli, and Debora Marks. RITA: a study on scaling up generative protein sequence models. In *ICML 2022 Workshop on Computational Biology*, 2022.
- [18] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [19] Guillaume Huguet, James Vuckovic, Kilian Fatras, Eric Thibodeau-Laufer, Pablo Lemos, Riashat Islam, Cheng-Hao Liu, Jarrod Rector-Brooks, Tara Akhound-Sadegh, Michael Bronstein, Alexander Tong, and Avishek Joey Bose. Sequence-augmented SE(3)-flow matching for conditional protein generation. In *Advances in Neural Information Processing Systems*, volume 37, pages 33007–33036, 2024. doi: 10.52202/079017-1039. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/39ca8893ea38905a9d2ffe786e85af0f-Paper-Conference.pdf.
- [20] John B Ingraham, Max Baranov, Zak Costello, Karl W Barber, Wujie Wang, Ahmed Ismail, Vincent Frappier, Dana M Lord, Christopher Ng-Thow-Hing, Erik R Van Vlack, et al. Illuminating protein space with a programmable generative model. *Nature*, 623(7989):1070–1078, 2023. doi: 10.1038/s41586-023-06728-8.
- [21] Sarthak Jain, Joel Beazer, Jeffrey A. Ruffolo, Aadyot Bhatnagar, and Ali Madani. E1: Retrieval-augmented protein encoder models. *Technical report*, 2025. URL <https://storage.googleapis.com/e1-paper-a26c3c79/profluent-e1.pdf>.
- [22] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.
- [23] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [24] Xiaohan Lin, Zhenyu Chen, Yanheng Li, Zicheng Ma, Chuanliu Fan, Ziqiang Cao, Shihao Feng, Yi Qin Gao, and Jun Zhang. Tokenizing foldable protein structures with machine-learned artificial amino-acid vocabulary. *bioRxiv*, 2023. doi: 10.1101/2023.11.27.568722.
- [25] Yeqing Lin and Mohammed AlQuraishi. Generating novel, designable, and diverse protein structures by equivariantly diffusing oriented residue clouds. *arXiv preprint arXiv:2301.12485*, 2023. doi: 10.48550/arXiv.2301.12485.
- [26] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.
- [27] Sidney L Lisanza, Jacob M Gershon, Samuel W Tipps, Jeremiah N Sims, Lucas Arnoldt, Samuel J Hendel, Miriam K Simma, Ge Liu, Marioka Yase, Hong Wu, et al. Multistate and functional protein design using RoseTTAFold sequence space diffusion. *Nature Biotechnology*, 2025. doi: 10.1038/s41587-024-02395-w.

- [28] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.
- [29] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [30] Amy X. Lu, Wilson Yan, Vladimir Gligoričević, Kyunghyun Cho, Richard Bonneau, Kevin K. Yang, Pieter Abbeel, and Nathan C. Frey. Generating all-atom protein structure from sequence-only training data. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=6PEb111COM>.
- [31] Jiarui Lu, Xiaoyin Chen, Stephen Zhewen Lu, Chence Shi, Hongyu Guo, Yoshua Bengio, and Jian Tang. Structure language models for protein conformation generation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [32] Tianyu Lu, Richard Shuai, Petr Kouba, Zhaoyang Li, Yilin Chen, Akio Shirali, Jinho Kim, and Po-Ssu Huang. Conditional protein structure generation with Protpardelle-1c. *bioRxiv*, 2025. doi: 10.1101/2025.08.18.670959.
- [33] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
- [34] Erik Nijkamp, Jeffrey A Ruffolo, Eli N Weinstein, Nikhil Naik, and Ali Madani. ProGen2: Exploring the boundaries of protein language models. *Cell Systems*, 14(11):968–978, 2023. doi: 10.1016/j.cels.2023.10.002.
- [35] OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [37] Mahdi Pourmirzaei, Farzaneh Esmaili, Salhuldin Alqarghuli, Mohammadreza Pourmirzaei, Ye Han, Kai Chen, Mohsen Rezaei, Duolin Wang, and Dong Xu. Prot2Token: A unified framework for protein modeling via next-token prediction. *arXiv preprint arXiv:2505.20589*, 2025.
- [38] Mahdi Pourmirzaei, Alex Morehead, Farzaneh Esmaili, Jarett Ren, Mohammadreza Pourmirzaei, and Dong Xu. GCP-VQVAE: A geometry-complete language for protein 3D structure. *bioRxiv*, 2025. doi: 10.1101/2025.10.01.679833. Version 3.
- [39] Wei Qu, Jiawei Guan, Rui Ma, Ke Zhai, Weikun Wu, and Haobo Wang. P(all-atom) is unlocking new path for protein design. In *International Conference on Machine Learning (ICML)*, 2025. URL <https://openreview.net/forum?id=yXRixu00NY>.
- [40] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI technical report*, 2019.

- [41] Vivek Ramanujan, Kushal Tirumala, Armen Aghajanyan, Luke Zettlemoyer, and Ali Farhadi. When worse is better: Navigating the compression-generation tradeoff in visual tokenization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [42] Chaofan Tao, Qian Liu, Longxu Dou, Niklas Muennighoff, Zhongwei Wan, Ping Luo, Min Lin, and Ngai Wong. Scaling laws with vocabulary: Larger models deserve larger vocabularies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [43] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, 53(D1):D609–D617, 2025. doi: 10.1093/nar/gkae1010.
- [44] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual autoregressive modeling: Scalable image generation via next-scale prediction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [45] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/7a98af17e63a0ac09ce2e96d03992fbc-Abstract.html>.
- [46] Michel van Kempen, Stephanie S. Kim, Charlotte Tumescheit, Milot Mirdita, Jeongjae Lee, Cameron L. M. Gilchrist, Johannes Söding, and Martin Steinegger. Fast and accurate protein structure search with Foldseek. *Nature Biotechnology*, 42:243–246, 2024. doi: 10.1038/s41587-023-01773-0.
- [47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [48] Xinyou Wang, Zaixiang Zheng, Fei Ye, Dongyu Xue, Shujian Huang, and Quanquan Gu. Diffusion language models are versatile protein learners. In *International Conference on Machine Learning (ICML)*, 2024.
- [49] Xinyou Wang, Zaixiang Zheng, Fei Ye, Dongyu Xue, Shujian Huang, and Quanquan Gu. DPLM-2: A multimodal diffusion protein language model. In *International Conference on Learning Representations (ICLR)*, 2025.
- [50] Joseph L Watson, David Juergens, Nathaniel R Bennett, Brian L Trippe, Jason Yim, Helen E Eisenach, Woody Ahern, Andrew J Borst, Robert J Ragotte, Lukas F Milles, et al. De novo design of protein structure and function with RFdiffusion. *Nature*, 620(7976):1089–1100, 2023. doi: 10.1038/s41586-023-06415-8.
- [51] Kevin K. Yang. Masked inverse folding with sequence transfer for protein representation learning. *Protein Engineering, Design and Selection*, 36:gza015, 2023. doi: 10.1093/protein/gza015.
- [52] Fei Ye, Zaixiang Zheng, Dongyu Xue, Yuning Shen, Lihao Wang, Yiming Ma, Yan Wang, Xinyou Wang, Xiangxin Zhou, and Quanquan Gu. ProteinBench: A holistic evaluation of protein foundation models. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2409.06744>.
- [53] Qihang Yu, Mark Weber, Xueqing Deng, Xiaohui Shen, Daniel Cremers, and Liang-Chieh Chen. An image is worth 32 tokens for reconstruction and generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

- [54] Xinyu Yuan, Zichen Wang, Marcus D. Collins, and Huzefa Rangwala. Protein structure tokenization: Benchmarking and new recipe. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267, pages 73645–73670. PMLR, 2025.
- [55] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2022. doi: 10.1109/TASLP.2021.3129994.
- [56] Jun Zhan, Junqi Dai, Jiasheng Ye, Yunhua Zhou, Dahua Zhang, Zhigeng Liu, Xin Zhang, Ruibin Yuan, Ge Zhang, Linyang Li, Hang Yan, Jie Fu, Tao Gui, Tianxiang Xu, Yugang Zheng, and Xipeng Qiu. AnyGPT: Unified multimodal LLM with discrete sequence modeling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [57] Jiahao Zhang, Zeqing Zhang, Di Wang, and Lijie Hu. Controlling repetition in protein language models. *arXiv preprint arXiv:2602.00782*, 2026.
- [58] Jiayou Zhang, Barthélemy Meynard-Piganeau, James Gong, Xingyi Cheng, Yingtao Luo, Hugo Ly, Le Song, and Eric Xing. Balancing locality and reconstruction in protein structure tokenizer. In *NeurIPS 2024 Workshop on Machine Learning in Structural Biology (MLSB)*, 2024. doi: 10.1101/2024.12.02.626366. URL <https://www.biorxiv.org/content/10.1101/2024.12.02.626366v2>.

Appendix

TABLE OF CONTENTS

A	Methods	21
A.1	PLL Tokenizer Details	21
A.2	PLL Metrics	22
A.3	SLL Tokenizer Details	23
A.4	SLL Metrics	24
A.5	Autoregressive Tokenization Protocol	25
A.6	Confidence-Based Candidate Selection	26
A.7	Scaling-Law Estimation	27
A.8	Sequence-Side De Novo Evaluation	28
A.9	Structure-Side De Novo Evaluation	30
B	Data Sources	33
C	Additional Experimental Results	33
C.1	PLL Per-Amino-Acid Recovery	33
C.2	SLL	34
C.2.1	SLL Supervised PST Ablation	34
C.2.2	SLL Supervised PST Full-Run	34
C.2.3	SLL Reconstruction Comparison	34
C.2.4	SLL Unsupervised PST Full-Run	34
C.2.5	SLL Sequence-to-Structure Compatibility	35
C.3	Autoregressive Protein Language Models	35
C.3.1	PLLM Unconditional Generation	35
C.3.2	PLLM Conditional Generation	36
C.3.3	SLLM Length-Conditioned Continuation	38
C.3.4	SLLM Sequence-to-Structure Model Selection	38

A. Methods

A.1. PLL Tokenizer Details

PLL Stage 1 uses frozen ESM-2 representations as the target semantic space. For a valid residue position t , ESM-2 produces $\mathbf{h}_t$. A trainable linear projection, transformer encoder, and projection head produce the vector that enters the vector-quantization layer,

$$\mathbf{r}_{1:L} = E_\theta(W_{\text{in}}\mathbf{h}_{1:L}), \quad \mathbf{u}_t = W_{\text{vq}}\mathbf{r}_t. \quad (3)$$

The vector-quantization layer uses a single Euclidean codebook with one code per residue position. It computes the distance scores in Equation (1) only on valid residue positions. Padding positions are masked and do not contribute to assignment statistics or losses.

During Stage 1 training, PLL uses hard stochastic code assignment from the same scores,

$$\mathbf{z}_t^{\text{PLL}} = \arg \max_k \left(\frac{\ell_{t,k}}{\tau} + g_{t,k} \right), \quad g_{t,k} \sim \text{Gumbel}(0, 1), \quad \mathbf{q}_t = \mathbf{c}_{\mathbf{z}_t^{\text{PLL}}}. \quad (4)$$

The temperature is fixed to $\tau = 0.1$ in Stage 1. At evaluation, decoding, and Stage 2 tokenization, the Gumbel noise is removed and the assignment reduces to the deterministic nearest-code rule in Equation (1).

The codebook is initialized by k-means and updated by exponential moving-average (EMA) statistics rather than direct gradient descent. Let ρ denote the EMA decay, and let $m_{t,k}$ be the one-hot assignment for valid position t . For each code k , EMA updates assignment counts and embedding sums as

$$n_k \leftarrow \rho n_k + (1 - \rho) \sum_t m_{t,k}, \quad \mathbf{e}_k \leftarrow \rho \mathbf{e}_k + (1 - \rho) \sum_t m_{t,k} \mathbf{u}_t, \quad \mathbf{c}_k \leftarrow \frac{\mathbf{e}_k}{\tilde{n}_k}, \quad (5)$$

where $\tilde{n}_k$ is the EMA-smoothed assignment count. In the trained PLL tokenizer, $\rho = 0.99$. Rare codes below the dead-code threshold are replaced from current batch samples, which helps maintain codebook coverage. For the final tokenizer, the VQ layer uses k-means initialization with 10 iterations, EMA decay 0.99, and a dead-code threshold of two. Stage 1 uses stochastic code assignment with temperature 0.1. Stage 2 uses deterministic assignments from the frozen Stage 1 codebook.

The vector-quantization contribution to the Stage 1 objective is the commitment loss under the hard assignment above, while codebook locations are maintained by k-means initialization with 10 iterations, EMA updates with decay 0.99, and dead-code replacement with threshold two.

For valid residue positions T , the Stage 1 terms in Equation (2) are

$$\begin{aligned} \mathcal{L}_{\text{mse}} &= \frac{1}{|T|} \sum_{t \in T} \|\hat{\mathbf{h}}_t - \mathbf{h}_t\|_2^2, \\ \mathcal{L}_{\text{cos}} &= \frac{1}{|T|} \sum_{t \in T} \left(1 - \frac{\langle \hat{\mathbf{h}}_t, \mathbf{h}_t \rangle}{\|\hat{\mathbf{h}}_t\|_2 \|\mathbf{h}_t\|_2} \right), \\ \mathcal{L}_{\text{commit}} &= \frac{1}{|T|} \sum_{t \in T} \|\mathbf{u}_t - \text{sg}(\mathbf{q}_t)\|_2^2. \end{aligned} \quad (6)$$

where $\text{sg}(\cdot)$ stops gradients through the selected code vector. Thus, $\mathcal{L}_{\text{rec}} = 10 \mathcal{L}_{\text{mse}} + 1 \mathcal{L}_{\text{cos}}$ and $\mathcal{L}_{\text{vq}} = 0.5 \mathcal{L}_{\text{commit}}$ in Equation (2). The MSE and cosine terms reconstruct the frozen ESM-2 representation, while the commitment term trains the encoder outputs to stay close to their assigned code vectors.

PLL Stage 1 keeps ESM-2 150M fixed and learns a symmetric latent encoder-decoder around a residue-wise VQ codebook. The latent encoder and decoder each use 12 transformer layers, width 768, 12 attention heads, 64-dimensional attention heads, three key-value heads, and a feedforward multiplier of four. The VQ codebook contains 4096 codes of dimension 128. In Stage 2, the Stage 1 decoder weights are discarded and the transformer decoder is reinitialized for residue classification. The fixed 128-dimensional quantized vectors are projected to 768 dimensions, processed by the 12-layer transformer decoder, and mapped to 21 residue classes by a linear output head. Stage 1 trains the latent encoder, VQ layer, and latent decoder. Stage 2 freezes the latent encoder and VQ layer and trains the reinitialized transformer decoder, its input projection, and the residue-classification head.

Table 6: PLL training choices for the two tokenizer stages. Stage 1 learns the latent bottleneck, and Stage 2 freezes the latent encoder and codebook and trains the transformer decoder for residue recovery. Maximum sequence length includes the beginning-of-sequence and end-of-sequence tokens from the ESM tokenizer.

Setting	Stage 1	Stage 2
Objective	$\mathcal{L}_{\text{mse}}, \mathcal{L}_{\text{cos}}, \mathcal{L}_{\text{commit}}$	$\mathcal{L}_{\text{class}}$
Epochs	64	8
Maximum sequence length	1 024	1 024
Training size	2.0M	20.0M
Batch size	512 (16 per GPU)	2 048 (64 per GPU)
Optimizer	AdamW, weight decay 0.01, gradient clipping	Same as Stage 1
	1.0	
Learning-rate range	1×10^{-4} to 1×10^{-6}	Same as Stage 1
Schedule	4 096 warmup steps, cosine decay	Same as Stage 1

A.2. PLL Metrics

PLL evaluation follows the two-stage training procedure. In Stage 1, the tokenizer is evaluated as a latent autoencoder. A sequence is encoded by frozen ESM-2, passed through the quantized bottleneck, and decoded back to ESM-2 space. Let $\mathbf{h}_{s,t}$ and $\hat{\mathbf{h}}_{s,t}$ denote the original and reconstructed ESM-2 vectors for residue t in sequence s , and let T_s be the valid residue positions. The Stage 1 metrics measure how faithfully this process reconstructs the ESM-2 latent representation.

The $\Delta\text{PPL}_{\%}$ metric, or percent delta perplexity, is the primary Stage 1 monitoring target. It converts the original and reconstructed vectors into distributions and reports the average relative increase in latent perplexity after reconstruction. Let $\text{CE}_s^{\text{base}}$ be the self-entropy of the original latent distribution and CE_s^{rec} be the cross-entropy from the original distribution to the reconstructed one. We compute

$$\Delta\text{PPL}_{\%} = 100 \left(\frac{1}{N} \sum_{s=1}^N \exp(\text{CE}_s^{\text{rec}} - \text{CE}_s^{\text{base}}) - 1 \right). \quad (7)$$

Lower values mean that the reconstructed representation behaves more like the original ESM-2 representation under this latent-distribution comparison.

Reconstruction Kullback-Leibler (KL) divergence measures the distributional gap between the same original and reconstructed latent vectors after softmax normalization.

$$\text{KL}_{\text{rec}} = \frac{1}{N} \sum_{s=1}^N \frac{1}{|T_s|} \sum_{t \in T_s} D_{\text{KL}} \left(\text{softmax}(\mathbf{h}_{s,t}) \parallel \text{softmax}(\hat{\mathbf{h}}_{s,t}) \right). \quad (8)$$

Median normalized mean squared error (NMSE) gives the complementary vector-space view, normalizing each sequence-level squared error by the energy of the original latent representation.

$$\text{NMSE}_{\text{median}} = \text{median}_s \frac{\sum_{t \in T_s} \|\mathbf{h}_{s,t} - \hat{\mathbf{h}}_{s,t}\|_2^2}{\sum_{t \in T_s} \|\mathbf{h}_{s,t}\|_2^2}. \quad (9)$$

The median reports typical reconstruction quality without letting unusually difficult sequences dominate the summary.

In Stage 2, the learned tokenizer is fixed and the evaluation asks whether PLL codes retain enough information to recover amino acid identities. The residue vocabulary contains 21 classes, consisting of the 20 standard amino acids and X, which represents unknown or non-standard residues. The primary metric is micro token-level amino acid recovery accuracy.

$$\text{Acc}_{\text{micro}} = \frac{\sum_s \sum_{t \in T_s} \mathbf{1}[\arg \max_c p_\phi(c \mid z_{s,1:L_s}^{\text{PLL}}, t) = x_{s,t}]}{\sum_s |T_s|}. \quad (10)$$

This is the overall recovery rate over valid residues. Padding positions are ignored, special tokens are not included, and unknown or non-standard residues are mapped to the valid class X. Per-amino-acid recovery accuracy repeats the same calculation within each residue class.

$$\text{Acc}(a) = \frac{\sum_s \sum_{t \in T_s} \mathbf{1}[x_{s,t} = a] \mathbf{1}[\arg \max_c p_\phi(c \mid z_{s,1:L_s}^{\text{PLL}}, t) = a]}{\sum_s \sum_{t \in T_s} \mathbf{1}[x_{s,t} = a]}. \quad (11)$$

This class-wise view checks whether high micro accuracy is shared across residue types rather than driven only by frequent amino acids. Stage 1 reconstruction metrics and Stage 2 residue recovery are tokenizer diagnostics. They are distinct from the downstream next-token validation perplexity reported for the SLL sequence-to-structure comparison in Section C.2.5.

A.3. SLL Tokenizer Details

SLL uses the GCP-VQVAE Lite [38] architecture as its backbone tokenizer. The input is a protein backbone represented by residue-level geometry. The GCPNet encoder uses chirality-sensitive message passing with rotation- and translation-invariant scalar features and rotation-equivariant vector features [38]. Its rotation- and translation-invariant scalar residue readouts are contextualized by a bidirectional transformer encoder before quantization. The quantizer assigns one discrete SLL code per valid residue position from a codebook of size 4096. Padding and invalid coordinate positions are masked before loss and metric computation.

The decoder receives the quantized residue sequence and reconstructs the three-atom backbone coordinates (N, C_α, C) through a transformer decoder and six-dimensional rotation head. The final reconstruction objective contains an aligned coordinate term and a pairwise backbone-distance term. For the aligned coordinate term, target coordinates are Kabsch-aligned to the predicted coordinates over valid residues before computing elementwise mean squared error. For the distance term, pairwise distances are computed over valid backbone atoms and compared between the predicted and target structures. The vector-quantization loss keeps encoder outputs close to their assigned code vectors. The coordinate objective compares backbones up to a global rigid transformation, while the distance objective is invariant to global translation and rotation. Vector quantization can still introduce internal geometric reconstruction error. For a structure s , let T_s be the valid residue positions and let $\mathcal{A}_s$ be the corresponding valid backbone atoms. Let $\hat{\mathbf{y}}_{s,t,a}$ be the predicted

coordinate for atom a at residue t , and let $\tilde{\mathbf{y}}_{s,t,a}$ be the Kabsch-aligned target coordinate. Let $\hat{\mathcal{V}}_s = \{\hat{\mathbf{v}}_{s,m}\}_{m=1}^{6|T_s|}$ and $\tilde{\mathcal{V}}_s = \{\tilde{\mathbf{v}}_{s,m}\}_{m=1}^{6|T_s|}$ be the predicted and aligned-target backbone direction and normal vectors used by the GCP-VQVAE direction loss. The reconstruction terms considered in our tokenizer study are defined per structure and averaged over the batch,

$$\begin{aligned}\mathcal{L}_{\text{coord}} &= \frac{1}{3|\mathcal{A}_s|} \sum_{(t,a) \in \mathcal{A}_s} \|\hat{\mathbf{y}}_{s,t,a} - \tilde{\mathbf{y}}_{s,t,a}\|_2^2, \\ \mathcal{L}_{\text{dist}} &= \frac{1}{|\mathcal{A}_s|^2} \sum_{i,j \in \mathcal{A}_s} \min\left((\|\hat{\mathbf{y}}_{s,i} - \hat{\mathbf{y}}_{s,j}\|_2 - \|\tilde{\mathbf{y}}_{s,i} - \tilde{\mathbf{y}}_{s,j}\|_2)^2, 25\right), \\ \mathcal{L}_{\text{dir}} &= \frac{1}{(6|T_s|)^2} \sum_{m,n=1}^{6|T_s|} \min\left((\langle \hat{\mathbf{v}}_{s,m}, \hat{\mathbf{v}}_{s,n} \rangle - \langle \tilde{\mathbf{v}}_{s,m}, \tilde{\mathbf{v}}_{s,n} \rangle)^2, 20\right).\end{aligned}\tag{12}$$

The distance term clips each squared error at 25. The direction term compares pairwise dot products among local backbone direction and normal vectors and clips each squared error at 20. When enabled in reconstruction-loss ablations, it enters with coefficient 0.05. It is not used in the final SLL tokenizer.

The SLL-specific change is the addition of semantic auxiliary heads during tokenizer training. Two heads act before the structure decoder and therefore supervise the quantized stream directly. The inverse-folding head predicts the 21-class residue identity vocabulary, consisting of the 20 standard amino acids and X. The pLDDT head predicts the per-residue confidence target. A third head acts after the decoder and predicts frozen ESM-2 representations. Let $\mathbf{q}_{s,t}$ be the quantized vector at residue t , let $r_{s,t}$ be the pLDDT target, and let $\mathbf{e}_{s,t}$ be the frozen ESM-2 target representation. Let $T_s^{\text{pLDDT}} \subseteq T_s$ be the valid positions with observed pLDDT targets, and let d_{ESM} be the ESM-2 representation dimension. The auxiliary losses are

$$\begin{aligned}\mathcal{L}_{\text{if}} &= \frac{1}{|T_s|} \sum_{t \in T_s} \text{CE}(x_{s,t}, p_{\text{if}}(\cdot | \mathbf{q}_{s,t})), \\ \mathcal{L}_{\text{pLDDT}} &= \frac{1}{|T_s^{\text{pLDDT}}|} \sum_{t \in T_s^{\text{pLDDT}}} |\hat{r}_{s,t} - r_{s,t}|, \\ \mathcal{L}_{\text{ESM}} &= \frac{1}{d_{\text{ESM}}|T_s|} \sum_{t \in T_s} \|\hat{\mathbf{e}}_{s,t} - \mathbf{e}_{s,t}\|_2^2.\end{aligned}\tag{13}$$

These auxiliary terms are also averaged over the batch before weighting. Using the final SLL tokenizer coefficients, the objective combines the active reconstruction, quantization, and auxiliary terms,

$$\mathcal{L}_{\text{SLL}} = 0.002 \mathcal{L}_{\text{coord}} + 0.01 \mathcal{L}_{\text{dist}} + \mathcal{L}_{\text{vq}} + 0.02 \mathcal{L}_{\text{if}} + 0.1 \mathcal{L}_{\text{pLDDT}} + 0.5 \mathcal{L}_{\text{ESM}}.\tag{14}$$

The selected SLL recipe drops $\mathcal{L}_{\text{dir}}$ and keeps the coordinate and pairwise-distance reconstruction losses in Equation (14). The auxiliary heads are removed from the autoregressive generation interface. After tokenizer training, an SLL sequence is simply a sequence of structure-code IDs, and generated SLL tokens are decoded with the fixed SLL decoder.

A.4. SLL Metrics

This subsection defines the reconstruction metrics used to verify that SLL codes remain decodable to backbone coordinates. Representation-quality evaluation follows the PST protocol from StructTokenBench [54]. The

Table 7: SLL tokenizer training choices for the final GCP-VQVAE Lite 2 model. The architecture and source dataset follow GCP-VQVAE Lite.

Setting	Value
Active objective terms	$\mathcal{L}_{\text{coord}}, \mathcal{L}_{\text{dist}}, \mathcal{L}_{\text{vq}}, \mathcal{L}_{\text{if}}, \mathcal{L}_{\text{pLDDT}}, \mathcal{L}_{\text{ESM}}$
Epochs	64
Maximum sequence length	1 280 residues
Training size	2.28M
Batch size	512 (16 per GPU)
Precision	bfloat16
Optimizer	AdamW, weight decay 0.01, gradient clipping 1.0
Learning-rate range	1×10^{-4} to 1×10^{-5}
Schedule	4096 warmup steps, cosine decay
Coordinate augmentation	Random rigid rotations, Gaussian jitter with standard deviation 0.05, and coordinate masking

primary reconstruction metrics are TM-score and RMSD, computed after structural alignment between the reconstructed and target backbones. TM-score measures global fold similarity with length normalization, where higher is better. RMSD measures aligned coordinate error in angstroms, where lower is better.

During tokenizer training, we also monitor coordinate-space diagnostics on valid residues. Mean absolute error and RMSD summarize pointwise coordinate recovery, while GDT-TS and TM-score summarize structure-level agreement.

A.5. Autoregressive Tokenization Protocol

All autoregressive experiments use residue-wise tokenization for amino acid, PLL, and SLL streams. Each valid residue position contributes one content token in the selected language. The token inventory used in the experiments is summarized in Table 8. During cross-entropy training, targets set to <PAD> are ignored. Content tokens and boundary tokens are predicted normally, while condition-prefix targets such as <LEN_n> and <sequence_to_structure> are excluded from the loss.

Table 8: Token inventory used by the autoregressive tokenizer.

Type	Tokens
Special	<PAD>, <BOS>, <EOS>, <MASK>, <UNK>
Amino acid	A C D E F G H I K L M N P Q R S T V W Y X
PLL	learned code IDs 0 to 4095
SLL	learned structure code IDs 0 to 4095
Length condition	<LEN_n>
Protein boundary	<BOP>, <EOP>
Structure boundary	<B03D>, <E03D>
Sequence-to-structure task	<sequence_to_structure>

Examples of the resulting streams are shown in Table 9. Unconditional tokenization uses <BOS> content <EOS> for amino acid, PLL, and SLL language modeling. Conditional tokenization adds prefix or boundary tokens before the generated content. Length-conditioned continuation uses <LEN_n> as an instruction, where n is the content length after truncation. For sequence-to-structure, the amino acid sequence is encoded outside the causal token stream by the E1 protein encoder, and the decoder predicts SLL tokens autoregressively. All streams are padded to the model context length, and padded targets do not contribute to loss or perplexity.

Table 9: Example token streams for the autoregressive experiments. P_i denotes a PLL code token, and S_i denotes an SLL code token.

Case	Stream format
Amino acid	$\langle \text{BOS} \rangle A C D \dots Y \langle \text{EOS} \rangle$
PLL	$\langle \text{BOS} \rangle P_1 P_2 \dots P_L \langle \text{EOS} \rangle$
SLL	$\langle \text{BOS} \rangle S_1 S_2 \dots S_L \langle \text{EOS} \rangle$
Amino acid length condition	$\langle \text{BOS} \rangle \langle \text{LEN_n} \rangle A C D \dots Y \langle \text{EOS} \rangle$
PLL length condition	$\langle \text{BOS} \rangle \langle \text{LEN_n} \rangle \langle \text{BOP} \rangle P_1 \dots P_L \langle \text{EOP} \rangle \langle \text{EOS} \rangle$
SLL length condition	$\langle \text{BOS} \rangle \langle \text{LEN_n} \rangle \langle \text{B03D} \rangle S_1 \dots S_L \langle \text{E03D} \rangle \langle \text{EOS} \rangle$
Sequence-to-structure	$\langle \text{BOS} \rangle \langle \text{sequence_to_structure} \rangle \langle \text{B03D} \rangle S_1 \dots S_L \langle \text{E03D} \rangle \langle \text{EOS} \rangle$

A.6. Confidence-Based Candidate Selection

DeepConf [11] uses model-internal token statistics to score multiple generated traces and filter low-confidence outputs. We adapt this idea to sequence-to-structure generation in an offline setting, where all candidate SLL streams for an input sequence are generated before selection. This differs from online DeepConf because we do not stop generation early, and we do not vote over final answers. The selector chooses one structure-token sequence directly.

For an input sequence x , let $\mathbf{z}^{(j)} = (z_1^{(j)}, \dots, z_{L_j}^{(j)})$ be candidate j among k sampled SLL streams. At generated structure-token position t , the SLLM defines $p_\theta(\cdot \mid x, \mathbf{z}_{<t}^{(j)})$ over the SLL vocabulary. We measure internal uncertainty before coordinate decoding by the token entropy of the raw model distribution,

$$H_t^{(j)} = - \sum_{v \in \mathcal{V}_{\text{SLL}}} p_\theta(v \mid x, \mathbf{z}_{<t}^{(j)}) \log p_\theta(v \mid x, \mathbf{z}_{<t}^{(j)}). \quad (15)$$

Lower entropy indicates higher model confidence in the latent structure-token prediction. The confidence score is computed from the full vocabulary distribution before top- p filtering and before any temperature rescaling used for sampling.

Following the local-confidence view in DeepConf, we summarize candidate quality with local windows rather than only a full-sequence average. For a window G_r over generated positions, define

$$\bar{H}_r^{(j)} = \frac{1}{|G_r|} \sum_{t \in G_r} H_t^{(j)}. \quad (16)$$

The main selector scores a candidate by its least confident local region,

$$s_{\text{conf}}^{(j)} = \max_r \bar{H}_r^{(j)}, \quad (17)$$

and lower scores are preferred. This is the entropy-space analog of DeepConf’s lowest group confidence. We also use mean, tail, and bottom-percent group entropy as diagnostic variants during selection analysis.

When consensus selection is enabled, we first retain the most confident candidates under $s_{\text{conf}}^{(j)}$. If only one candidate is retained, selection is trivial. Among the retained set $\mathcal{R}$, we select the token-space medoid,

$$j^* = \arg \max_{j \in \mathcal{R}} \frac{1}{|\mathcal{R}| - 1} \sum_{\ell \in \mathcal{R}, \ell \neq j} \text{sim}(\mathbf{z}^{(j)}, \mathbf{z}^{(\ell)}), \quad (18)$$

where sim is the normalized position-wise overlap between two SLL token streams,

$$\text{sim}(\mathbf{z}, \mathbf{z}') = \frac{1}{\max(L, L')} \sum_{t=1}^{\min(L, L')} \mathbf{1}[z_t = z'_t]. \quad (19)$$

Here, L and L' are the lengths of $\mathbf{z}$ and $\mathbf{z}'$. The selected stream $\mathbf{z}^{(j^*)}$ is then decoded into coordinates by the fixed SLL decoder.

Oracle best-of- k is reported only as a ceiling on the sampled model’s performance. It selects

$$j_{\text{oracle}}^* = \arg \max_{1 \leq j \leq k} Q\left(D_{\text{SLL}}(\mathbf{z}^{(j)}), y\right), \quad (20)$$

where D_{SLL} is the SLL coordinate decoder, y is the native structure, and Q is a ground-truth structure-quality metric such as TM-score. Because this rule uses the native structure, it is not available at inference time. It measures how much quality exists inside the sampled candidate pool and gives a ceiling for confidence-based selection.

A.7. Scaling-Law Estimation

The scaling analysis compares validation cross-entropy loss against compute for the amino acid model, PLLM, and SLLM. For each token space, six completed training runs are selected for the fit. For each validation point i , cumulative compute is estimated from dense padded non-embedding training FLOPs,

$$C_i = \frac{s_i F_{\text{step}}}{10^{15}}, \quad (21)$$

where s_i is the optimizer step and F_{step} is the per-step training FLOP estimate. We use the standard forward-plus-backward convention of three times the forward FLOPs [18], exclude embedding lookups, and report C_i in PFLOPs. These estimates cover downstream autoregressive training and exclude encoder pretraining, tokenizer learning, and preprocessing of the tokenized corpora. The amino acid-PLL match concerns this downstream stage. Total representation-building resources are not matched.

For the compute-scaling fit, validation points are sorted by C_i and reduced to the lower-loss frontier. We then fit a zero-offset power law,

$$L(C) = AC^{-\alpha}, \quad (22)$$

by ordinary least squares in log space,

$$\log L_i = \log A - \alpha \log C_i + \epsilon_i. \quad (23)$$

The reported scaling exponent is α . These exponents summarize the fitted lower-loss frontiers under the zero-offset model. Fit confidence intervals and sensitivity to an additive irreducible-loss term are not reported.

Table 10 gives the shared transformer grid used for all three token spaces. All models use tied input and output embeddings and rotary positional embeddings. At each grid point, the transformer blocks have the same dimensions and attention configuration. Vocabulary size changes the size of the tied embedding/output matrix, output-projection computation, and total parameter count.

Table 11 summarizes the training settings. The amino acid model and PLLM use the same UniRef50 sequences and optimization settings, each seeing approximately 70.7B tokens over 4 epochs. SLLM uses AFDB $\cap$ UniRef50 structure-token data and the same transformer architecture grid and next-token objective, but sees approximately 81.5B tokens over 8 epochs. The minimum learning rate is 1×10^{-5} for the amino acid model and PLLM and 1×10^{-6} for SLLM.

Table 10: Decoder-only transformer grid used for the scaling-law runs. The head dimension is 64 throughout.

Size label	Width	Layers	Heads	KV heads	Head dim	FFN dim
17.9M	512	6	4	4	64	2 048
51.8M	640	12	5	5	64	2 560
109.7M	768	19	6	3	64	3 072
263.4M	1 024	26	8	4	64	4 096
611.8M	1 536	27	12	6	64	6 144
1.33B	2 048	34	16	4	64	8 192

Table 11: Training hyperparameters for the scaling-law runs.

Setting	Amino acid model and PLLM	SLLM
Training data	UniRef50	AFDB $\cap$ UniRef50 [38]
Epochs	4	8
Steps per epoch	128k	128k
Total tokens seen per model	70.7B	81.5B
Batch size	512 (16 per GPU)	512 (16 per GPU)
Precision	bfloat16	bfloat16
Optimizer	8-bit AdamW	8-bit AdamW
Learning-rate range	1×10^{-4} to 1×10^{-5}	1×10^{-4} to 1×10^{-6}
Weight decay	1×10^{-1}	1×10^{-1}
Adam parameters	$\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 1 \times 10^{-7}$	$\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 1 \times 10^{-7}$
Gradient clipping	1.0	1.0
Schedule	2 048 warmup steps, cosine decay	2 048 warmup steps, cosine decay

A.8. Sequence-Side De Novo Evaluation

The amino acid and PLLM de novo experiments are evaluated in the same decoded residue space. The amino acid model generates residue tokens directly, while PLLM generates PLL code IDs that are decoded back to amino acid sequences with the fixed PLL decoder. For length-controlled generation, both models use the same length-token protocol described in Section A.5.

We use amino acid composition entropy as a distributional diagnostic before structural evaluation. For a decoded sequence $x = (x_1, \dots, x_L)$, let

$$p_x(a) = \frac{1}{L} \sum_{t=1}^L \mathbf{1}[x_t = a], \quad H_{\text{seq}}(x) = - \sum_{a \in \mathcal{A}} p_x(a) \log_2 p_x(a), \quad (24)$$

where $\mathcal{A}$ is the decoded residue alphabet. This entropy is computed once per complete sequence and is not further divided by sequence length or normalized by $\log_2 |\mathcal{A}|$. We define a generated sequence as drifted when $H_{\text{seq}}(x) < 1.5$ bits. The 1.5-bit threshold is a heuristic for low-complexity residue composition. Since H_{seq} depends only on residue frequencies, it does not measure residue order, structural quality, or protein function. We use it to compare amino acid and PLLM generation in the same decoded residue space. For the sequence-level ProteinBench comparison [52], we retain sequences with $H_{\text{seq}}(x) > 1.5$ bits and evaluate 50 folded sequences per model, target length, and sampling temperature. Additional generation is used when needed to reach this retained count.

The low-entropy filter is a first-order fairness control, not a biological correctness label. pLDDT is a predicted local-confidence measure introduced with AlphaFold [22] and also reported by ESMFold [26]. It does not provide experimental validation of the structure or biological function of a generated sequence. Low-complexity protein regions are compositionally biased or repetitive and can have distinct structural

behavior [6]. In generated samples, repetitive low-entropy outputs can therefore make pLDDT partly reflect escape from residue-composition collapse rather than the quality of the full generator.

For sequence-level ProteinBench, generated sequences are evaluated along the quality, diversity, and novelty axes.

Quality. In this work, the sequence-side quality metric is the mean pLDDT of the predicted structure. If c_t is the per-residue pLDDT for residue t , then

$$\text{pLDDT}(x) = \frac{1}{L} \sum_{t=1}^L c_t. \quad (25)$$

Higher pLDDT indicates higher folding-model confidence. In our experiments, pLDDT is parsed from the ESMFold [26] structure predicted for each generated sequence. Because the generated object is a sequence rather than a backbone, there is no original generated structure to compare against, so sequence-level evaluation does not report scTM or scRMSD.

Diversity. Foldseek [46] is used to measure structural diversity over the folded generated sequences. For folded structures $\{g_i\}_{i=1}^n$, pairwise TM is the mean of the maximum query or target TM-score over unordered non-self pairs,

$$\text{pairwise TM} = \frac{2}{n(n-1)} \sum_{1 \leq i < j \leq n} \max\{\text{TM}_{i \rightarrow j}, \text{TM}_{j \rightarrow i}\}. \quad (26)$$

Lower pairwise TM indicates higher diversity. Max Clust. is the number of Foldseek clusters divided by n , so higher values indicate a less collapsed generated set.

Novelty. Novelty is measured by Max TM, the mean over generated structures of the best TM-score to the Protein Data Bank (PDB) reference database,

$$\text{Max TM} = \frac{1}{n} \sum_{i=1}^n \max_{r \in \mathcal{R}_{\text{PDB}}} \max\{\text{TM}_{i \rightarrow r}, \text{TM}_{r \rightarrow i}\}. \quad (27)$$

Lower Max TM indicates greater novelty relative to known PDB structures.

Table 12: Length-wise sequence ProteinBench. Rows marked ^{ref} are taken from ProteinBench [52]. Our PLM and PLLM use $T = 0.5$, retain 50 sequences with $H_{\text{seq}}(x) > 1.5$ bits per length, and use ESMFold.

Length	Model	pLDDT $\uparrow$	Pair. TM $\downarrow$	Clust. $\uparrow$	Max TM $\downarrow$
100	Native ^{ref}	68.46	0.5500	0.7500	–
100	ProGen2 (700M) ^{ref}	64.00	0.4200	0.9400	0.6400
100	DPLM (650M) ^{ref}	85.38	0.5000	0.8000	0.7400
100	ESM3 (1.4B) ^{ref}	54.26	0.4500	0.9000	0.6800
100	PLLM (1.3B)	42.74	0.2825	0.9800	0.4189
100	PLM (1.3B)	64.75	0.3019	0.8400	0.4741
200	Native ^{ref}	61.91	0.4900	0.7800	–
200	ProGen2 (700M) ^{ref}	69.91	0.4000	0.9100	0.6900
200	DPLM (650M) ^{ref}	93.54	0.5400	0.7000	0.9100
200	ESM3 (1.4B) ^{ref}	58.45	0.3500	1.0000	0.8000
200	PLLM (1.3B)	59.37	0.2674	0.9600	0.6126
200	PLM (1.3B)	69.02	0.2406	0.9400	0.6048
300	Native ^{ref}	61.49	0.5100	0.8500	–
300	ProGen2 (700M) ^{ref}	65.69	0.4200	0.9300	0.6600
300	DPLM (650M) ^{ref}	93.07	0.5700	0.6300	0.9100
300	ESM3 (1.4B) ^{ref}	48.08	0.3200	1.0000	0.7500
300	PLLM (1.3B)	69.21	0.2472	0.9800	0.7441
300	PLM (1.3B)	68.23	0.2222	0.9600	0.6419
500	Native ^{ref}	62.95	0.5100	0.7800	–
500	ProGen2 (700M) ^{ref}	61.45	0.3200	0.9500	0.6800
500	DPLM (650M) ^{ref}	87.73	0.4300	0.8500	0.8500
500	ESM3 (1.4B) ^{ref}	52.17	0.3000	1.0000	0.5400
500	PLLM (1.3B)	68.11	0.2297	0.9600	0.7714
500	PLM (1.3B)	59.03	0.2005	0.9600	0.6228

The main-table means in Table 4 give equal weight to the four length rows. ProteinBench specifies 50 sequences per reference model and target length but does not specify exact repeat counts or sequence-generation temperatures. Our reported subsets use selection seed 42. Additional generation under the same condition fills retained sets when needed. Our rows report these fixed-seed evaluation sets without uncertainty estimates across independent evaluation repeats.

A.9. Structure-Side De Novo Evaluation

SLLM de novo generation is evaluated as backbone generation. SLLM samples an SLL token sequence, and the fixed SLL decoder converts the generated code IDs into backbone coordinates. We then evaluate the decoded backbones with the structure-design ProteinBench protocol [52]. This path differs from the sequence-side protocol because the generated object is already a structure, so self-consistency metrics are defined. The same quality, diversity, and novelty axes are used. For length-conditioned SLLM generation, the largest SLLM is continued for two epochs with length prefixes on 50% of samples, using the tokenization protocol in Section A.5.

Quality. For each target length, we evaluate 100 generated backbones. For a generated backbone b_i , ProteinMPNN [8] designs candidate sequences and ESMFold [26] refolds those sequences. We retain the maximum scTM and independently minimum scRMSD across the candidate designs. If $d_{i,m}$ is the refolded structure for design m , then

$$m_i^* = \arg \max_m \text{TM}(b_i, d_{i,m}), \quad \text{scTM}_i = \text{TM}(b_i, d_{i,m_i^*}), \quad \text{scRMSD}_i = \min_m \text{RMSD}(b_i, d_{i,m}). \quad (28)$$

The maximum-scTM and minimum-scRMSD designs can differ. Per-length quality scores are means over the 100 backbones. For FoldFlow-2 and DPLM-2, ProteinMPNN produces eight sequences per backbone

at sampling temperature 0.1. Higher scTM and lower scRMSD indicate that the generated backbone can support a sequence that refolds back to the same structure.

Diversity. Diversity is measured by nearest-neighbor TM and Max Clust. on the maximum-scTM refold set. For the $N = 100$ selected refolds, let $s_{ij} = \max\{\text{TM}_{i \rightarrow j}, \text{TM}_{j \rightarrow i}\}$. Nearest-neighbor TM is

$$\text{NN TM} = \frac{1}{N} \sum_{i=1}^N \max_{j \neq i} s_{ij}. \quad (29)$$

Lower nearest-neighbor TM and higher Max Clust. indicate more diverse samples.

Novelty. Novelty uses the same Max TM definition as in Section A.8, with lower values indicating less similarity to PDB references. All evaluated backbones are normalized to backbone heavy atoms and grouped by target length. PLAID outputs with lengths 56, 112, 320, and 528 are clipped to the target lengths 50, 100, 300, and 500 before evaluation. The structure-design comparison in Table 5 reports unweighted means over $L \in \{50, 100, 300, 500\}$. DPLM-2 uses the 650M model in unconditional backbone-generation mode. Native, RFdiffusion, and Genie results are taken from ProteinBench and marked with an asterisk. Their TM similarity values use the published pairwise TM measure, while unstarred rows use the nearest-neighbor TM measure defined above.

Length-wise structure results. Tables 13 and 14 report the individual target lengths used in the main structure-design comparison. The main table gives equal weight to $L \in \{50, 100, 300, 500\}$.

Table 13: Structure-design results on ProteinBench [52] at target lengths 50 and 100. * Results taken from ProteinBench. [†] DPLM-2 jointly models amino acid sequences and structure tokens.

Length	Model	scTM $\uparrow$	scRMSD $\downarrow$	TM similarity $\downarrow$	Clust. $\uparrow$	Max TM $\downarrow$
50	Native*	0.91	0.74	0.29	0.66	N/A
50	Genie* [25]	0.57	3.12	0.32	0.90	0.57
50	RFdiffusion* [50]	0.95	0.45	0.58	0.67	0.65
50	Protpardelle [32]	0.9655	0.5785	0.7369	0.3200	0.7608
50	ProteinGenerator [27]	0.7984	1.2202	0.8938	0.1300	0.3468
50	PLAID [30]	0.8028	1.3188	0.7012	0.5600	0.6463
50	La-Proteina [15]	0.9840	0.3816	0.7989	0.3200	0.8458
50	Pallatom [39]	0.9739	0.4937	0.7209	0.3300	0.7683
50	FoldFlow-2 [19]	0.8866	0.9081	0.7565	0.3700	0.8385
50	DPLM-2 (650M) [†] [49]	0.8679	1.0511	0.7791	0.5800	0.8735
50	SLLM	0.6194	1.7590	0.6088	0.7300	0.3418
100	Native*	0.96	0.67	0.30	0.77	N/A
100	Genie* [25]	0.69	3.38	0.31	0.96	0.59
100	RFdiffusion* [50]	0.98	0.48	0.41	0.32	0.76
100	Protpardelle [32]	0.9685	0.8236	0.8277	0.1000	0.8280
100	ProteinGenerator [27]	0.9602	0.8865	0.6844	0.3900	0.7563
100	PLAID [30]	0.8464	1.7470	0.6608	0.5600	0.7596
100	La-Proteina [15]	0.9887	0.4611	0.6294	0.5600	0.7439
100	Pallatom [39]	0.9829	0.5576	0.5819	0.6200	0.6956
100	FoldFlow-2 [19]	0.9392	1.0276	0.6738	0.3400	0.7622
100	DPLM-2 (650M) [†] [49]	0.8720	1.4944	0.7370	0.4400	0.8995
100	SLLM	0.5805	2.5859	0.4957	0.9600	0.5593

Table 14: Structure-design results on ProteinBench [52] at target lengths 300 and 500. * Results taken from ProteinBench. [†] DPLM-2 jointly models amino acid sequences and structure tokens.

Length	Model	scTM $\uparrow$	scRMSD $\downarrow$	TM similarity $\downarrow$	Clust. $\uparrow$	Max TM $\downarrow$
300	Native*	0.97	0.82	0.28	0.77	N/A
300	Genie* [25]	0.27	20.37	0.23	1.00	0.30
300	RFdiffusion* [50]	0.96	1.03	0.36	0.65	0.64
300	Protpardelle [32]	0.9603	1.3968	0.7395	0.1500	0.8152
300	ProteinGenerator [27]	0.8926	2.0403	0.6409	0.2600	0.7158
300	PLAID [30]	0.7474	3.0922	0.5950	0.5700	0.7085
300	La-Proteina [15]	0.9870	0.6999	0.5477	1.0000	0.6765
300	Pallatom [39]	0.8612	1.9246	0.4091	1.0000	0.5665
300	FoldFlow-2 [19]	0.9483	1.4434	0.4652	0.9700	0.7266
300	DPLM-2 (650M) [†] [49]	0.9182	1.7516	0.6787	0.5900	0.9077
300	SLLM	0.5210	4.0381	0.3778	0.9800	0.6427
500	Native*	0.97	1.07	0.29	0.80	N/A
500	Genie* [25]	0.25	26.08	0.23	1.00	0.22
500	RFdiffusion* [50]	0.79	5.60	0.33	0.89	0.62
500	Protpardelle [32]	0.8212	3.4017	0.6867	0.3000	0.7823
500	ProteinGenerator [27]	0.3582	6.2083	0.4025	0.8900	0.5036
500	PLAID [30]	0.4563	5.4272	0.4043	0.9200	0.5647
500	La-Proteina [15]	0.9113	1.6057	0.5162	0.7400	0.6815
500	Pallatom [39]	0.3786	6.2550	0.3991	1.0000	0.4879
500	FoldFlow-2 [19]	0.7689	3.3023	0.4126	1.0000	0.7463
500	DPLM-2 (650M) [†] [49]	0.8759	2.3074	0.7085	0.5400	0.9256
500	SLLM	0.4354	5.2065	0.3408	1.0000	0.6708

B. Data Sources

The project uses three main data sources. UniRef50 [43] provides the sequence corpus for learning PLL and for sequence generation. AFDB representative structures provide the structure corpus for learning SLL. For SLLM generation, we use the larger $\text{AFDB} \cap \text{UniRef50}$ structure corpus introduced by GCP-VQVAE [38]. This corpus is suitable for generation because SLL codes are trained with an inverse-folding auxiliary objective and therefore retain sequence-identity information. Starting from UniRef50 sequence clusters limits sequence redundancy at the 50% identity level before the corresponding AFDB structures are used as structure-token training data.

Table 15: Main data sources.

Data source	Role	Approx. samples
UniRef50 [43]	PLL tokenizer; PLLM and amino acid generation	65.78M
AFDB representatives	SLL tokenizer	2.28M
$\text{AFDB} \cap \text{UniRef50}$ following GCP-VQVAE [38]	SLLM generation	38.15M

The held-out validation splits contain 10,000 UniRef50 sequences, 10,000 AFDB representative structures, and 5,000 $\text{AFDB} \cap \text{UniRef50}$ structure-token pairs.

The UniRef50 reference statistics used for sequence-generation analysis are shown in Figure 8.

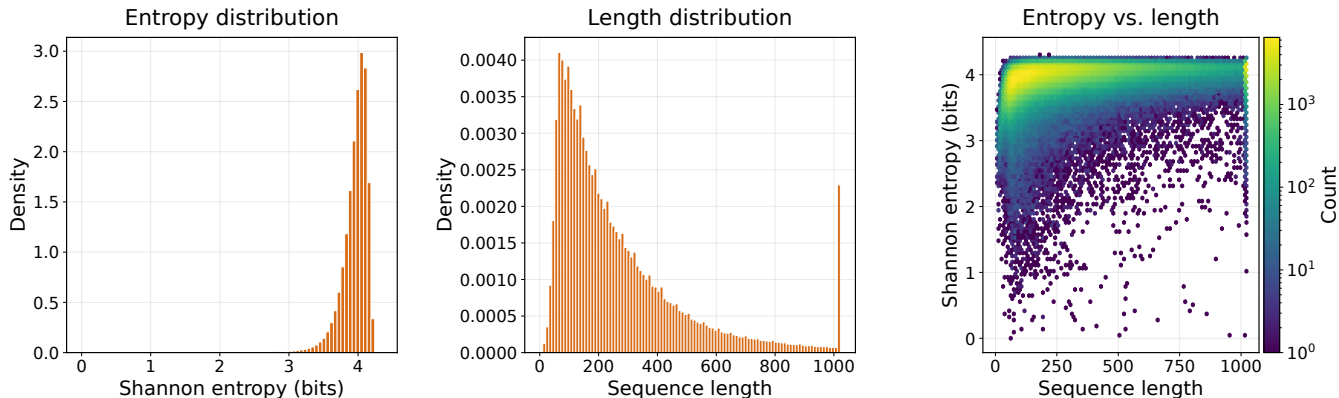


Figure 8: UniRef50 reference statistics after truncating sequences to 1022 residues. These distributions provide the reference length and amino acid composition statistics used when analyzing de novo amino acid and PLLM generation.

For SLL reconstruction comparisons, we use the benchmark splits and reporting protocol from GCP-VQVAE [38]. For representation selection, we use the PST protocol from StructTokenBench [54].

C. Additional Experimental Results

C.1. PLL Per-Amino-Acid Recovery

The Stage 2 PLL residue head reaches high recovery across all standard amino acids, not only in the micro average. Table 16 reports the validation accuracy for each residue class. The X class is lower because it represents unknown or non-standard residues rather than a single conventional amino acid.

Table 16: Stage 2 PLL per-amino-acid recovery on the UniRef50 validation split.

Residue	Accuracy (%)	Residue	Accuracy (%)	Residue	Accuracy (%)
A	99.89	I	99.81	R	99.81
C	99.18	K	99.77	S	99.87
D	99.83	L	99.91	T	99.86
E	99.87	M	99.48	V	99.87
F	99.72	N	99.79	W	99.20
G	99.86	P	99.84	Y	99.69
H	99.56	Q	99.80	X	90.52

C.2. SLL

C.2.1. SLL Supervised PST Ablation

Table 17 reports the supervised PST ablations used to select the final SLL recipe. Task columns average available splits, and the category average averages all rows within that category. Cell colors are relative to the Base tokenizer. The configuration names are shorthand for four groups of tokenizer changes.

Auxiliary heads. ESM pre-decoder and ESM post-decoder add frozen ESM-2 representation prediction before or after the coordinate decoder. Inverse folding 0.01, 0.1, and 1.0 add a pre-decoder amino acid recovery head with the indicated loss weight. pLDDT pre-decoder and pLDDT post-decoder add a per-residue confidence regression head at the indicated location.

Reconstruction losses. MSE, MSE + distance, MSE + direction, and Distance + direction change the active reconstruction losses. Distance denotes the pairwise backbone-distance loss, and direction denotes the GCP-VQVAE direction loss.

VQ choices. Rotation trick changes how gradients pass through the quantized vectors. The temperature rows test the codebook-temperature setting with or without the rotation trick.

Combined recipe. Base is the GCP-VQVAE Lite recipe under the same ablation protocol. The selected combined recipe uses MSE + distance reconstruction, inverse folding with weight 0.1, pLDDT pre-decoder, ESM post-decoder, and the rotation trick. The two-times-longer row repeats the same combined recipe for a longer ablation run.

C.2.2. SLL Supervised PST Full-Run

Table 18 follows the supervised PST table style used by the original work. The non-GCP baselines are reported with the PST-standard two-layer MLP probe. For GCP-VQVAE variants, parentheses report the four-layer MLP and transformer probes.

C.2.3. SLL Reconstruction Comparison

Table 19 reports the full reconstruction comparison behind the average results in Table 3. The baseline values follow the reconstruction table from the original work.

C.2.4. SLL Unsupervised PST Full-Run

Table 20 compares GCP-VQVAE Lite 2 with the original GCP-VQVAE tokenizers on unsupervised PST tasks. These scores are reported as secondary representation diagnostics for SLL.

Table 17: Supervised PST ablation results for SLL. Task columns average available splits, and Avg averages all rows within each category. Cells are colored relative to the Base row. Gray rows mark runs that did not converge cleanly.

Configuration	Functional AUROC (%) $\uparrow$								Physicochemical Spearman (%) $\uparrow$				
	BindInt	BindBio	BindShake	CatInt	CatBio	Con	Rep	Ept	Avg	RMSF	BFact	NEQ	Avg
Base	56.48	59.36	71.84	60.02	54.90	58.21	54.97	57.10	58.26	34.66	22.89	14.26	23.93
ESM pre-decoder	57.91	61.15	70.00	61.91	60.18	63.63	59.07	54.70	60.47	35.55	21.90	16.84	24.76
ESM post-decoder	58.94	61.86	73.03	63.11	58.12	61.18	57.07	57.88	60.62	36.70	24.20	14.88	25.26
Inverse folding 0.01	58.00	55.85	72.42	59.29	54.18	58.63	54.85	59.81	58.24	34.79	23.59	16.63	25.00
Inverse folding 1.0	67.18	88.90	63.38	63.06	86.18	62.00	55.07	58.48	68.34	25.96	17.15	21.49	21.53
Inverse folding 0.1	64.32	80.04	71.12	66.27	75.74	62.76	62.04	56.91	67.15	30.52	20.27	20.20	23.66
Distance + direction (not converged)	50.00	58.21	51.10	50.56	50.00	50.68	50.68	51.99	51.69	5.36	3.67	9.53	6.18
MSE	60.73	57.63	74.99	65.32	56.02	63.25	62.07	56.78	61.24	38.69	25.47	15.71	26.62
MSE + distance	58.78	61.98	76.92	62.56	56.85	62.08	58.75	60.14	61.28	44.13	29.29	17.55	30.32
MSE + direction	58.37	61.06	69.98	61.12	57.41	61.28	55.87	54.82	59.32	27.09	18.81	16.90	20.93
pLDDT post-decoder	59.75	64.39	73.10	63.40	61.77	60.75	57.98	59.47	61.87	36.98	25.51	16.52	26.33
pLDDT pre-decoder	58.94	71.42	71.82	60.89	70.26	59.65	54.22	58.59	62.65	41.34	25.43	20.08	28.95
ESM pre + pLDDT pre + IF 0.1	62.11	80.40	65.22	60.84	74.37	58.09	59.50	60.29	65.09	32.20	19.12	23.67	25.00
Rotation trick	60.95	75.85	73.12	63.28	70.42	61.13	59.62	56.45	64.56	34.61	22.81	16.00	24.47
Rotation trick + temp 0.1	59.77	62.98	72.88	61.70	60.58	59.36	56.81	56.17	60.50	36.04	23.82	14.07	24.64
Rotation trick + temp 0.5 (not converged)	59.47	64.34	55.82	56.53	65.01	59.39	59.77	53.80	59.49	17.40	10.18	13.95	13.84
Sample codebook temp 0.1	56.27	59.37	73.10	61.46	57.14	58.84	54.48	55.59	58.63	38.24	24.64	12.74	25.20
Sample codebook temp 0.5	57.57	63.71	68.13	60.80	58.36	58.27	55.14	57.61	59.40	29.00	18.72	13.22	20.31
Combined, selected	67.68	93.15	68.39	66.95	90.87	65.92	61.72	62.29	72.37	43.01	25.09	28.79	32.30
Combined, trained two times longer	67.14	93.09	72.97	68.49	92.16	67.30	63.25	63.50	73.52	39.36	24.91	28.35	30.87

C.2.5. SLL Sequence-to-Structure Compatibility

This experiment tests whether SLL is easier to predict from sequence under a next-token objective. We use a matched Prot2Token-style sequence-to-structure setup and change only the target structure tokenizer. Both runs use 200,000 AFDB representative training samples, 10,000 validation samples, and the same eight-epoch training budget. The sequence encoder, autoregressive decoder, data split, and optimizer setup are fixed across the two runs.

Figure 4 reports train loss and validation perplexity across epochs. The speed annotations summarize when GCP-VQVAE Lite 2 reaches comparable train-loss and validation-perplexity regimes relative to the original Lite tokenizer. Both runs reach their best validation perplexity at epoch four and then overfit, but GCP-VQVAE Lite 2 remains lower at every epoch. Its final validation perplexity, 22.16, is still below the best validation perplexity of the original Lite tokenizer, 22.68.

C.3. Autoregressive Protein Language Models

C.3.1. PLLM Unconditional Generation

Figure 9 shows the full entropy distribution sweep across sampling temperatures. The UniRef50 source distribution in Figure 8 is concentrated near four bits, and PLLM approaches this regime by $T = 0.6$ to $T = 1.0$. Direct amino acid generation remains lower-entropy across most of the sweep and only approaches the reference entropy range at much hotter sampling. At $T = 1.0$, PLLM reaches mean sequence entropy 3.927, close to the UniRef50 mean of 3.952, while the amino acid model remains lower at 3.331. Across the 12 unconditional sampling temperatures, 5 154/12 000 amino acid samples (43.0%) and 2 383/12 000 PLLM samples (19.9%) have $H_{\text{seq}}(x) < 1.5$ bits. This denominator includes the unconditional generations before folding or entropy filtering and is separate from the conditional evaluation populations.

Table 18: Supervised PST full-run comparison between SLL and structure-tokenizer baselines. GCP-VQVAE Lite 2 denotes SLL, and parentheses report four-layer MLP and transformer probe results for GCP-VQVAE variants.

Task	Split	FoldSeek [46]	ProTokens [24]	VanillaVQ [54]	AIDO [58]	ESM3 [16]	AminoAseed [54]	GCP-VQVAE Large [38]	GCP-VQVAE Lite [38]	GCP-VQVAE Lite 2
Functional Site Prediction, AUROC (%) $\uparrow$										
BindInt	Fold	53.18	44.66	47.25	44.66	44.30	47.11	52.61 (50.26, 52.85)	49.43 (49.00, 55.03)	52.60 (53.73, 50.85)
	SupFam	46.20	86.05	86.71	84.21	<u>90.77</u>	90.53	58.04 (70.61, 80.67)	62.27 (70.27, 80.77)	79.59 (79.84, 93.86)
BindBio	Fold	52.37	58.47	62.02	65.50	62.84	65.73	75.05 (92.74, 84.01)	80.93 (94.61 , 91.74)	93.97 (94.28, 87.99)
	SupFam	52.41	60.47	62.92	66.70	65.22	68.30	75.64 (93.40, 86.62)	81.72 (94.08, 90.08)	93.95 (94.10 , 88.52)
BindShake	Org	53.40	59.82	67.04	69.28	66.10	69.61	66.97 (67.96, <u>72.71</u>)	66.34 (67.11, 68.08)	70.81 (70.79, 75.75)
CatInt	Fold	53.43	58.16	58.89	57.30	61.09	<u>62.19</u>	55.92 (53.15, 60.67)	54.36 (55.49, 58.09)	53.46 (54.62, 66.03)
	SupFam	51.41	83.85	85.00	81.94	89.82	<u>91.91</u>	63.22 (73.43, 84.00)	62.24 (68.88, 85.18)	74.56 (71.92, 93.09)
CatBio	Fold	56.37	56.14	67.58	73.72	65.33	65.95	67.12 (79.66, 71.64)	67.25 (92.28 , 86.11)	90.90 (91.77, 71.61)
	SupFam	53.78	64.05	70.92	78.66	74.65	87.59	73.77 (89.64, 86.95)	75.57 (95.72 , 93.38)	94.19 (<u>94.88</u> , 89.89)
Con	Fold	49.26	56.23	56.98	56.64	55.22	57.23	52.64 (53.29, <u>62.97</u>)	51.15 (51.56, 61.52)	53.73 (53.08, 65.92)
	SupFam	51.39	74.33	74.60	73.79	80.53	<u>86.60</u>	61.31 (68.84, <u>84.05</u>)	58.66 (65.23, 85.79)	71.52 (72.25, 96.21)
Rep	Fold	47.70	<u>77.25</u>	75.99	77.69	74.70	74.97	51.19 (50.95, 66.19)	52.04 (52.79, 61.22)	48.81 (50.13, 74.85)
	SupFam	52.53	78.90	82.09	78.08	82.36	84.57	54.17 (65.26, <u>85.77</u>)	56.50 (54.65, 85.94)	72.89 (75.04, 85.26)
Ept	Fold	54.52	54.69	59.28	60.26	63.69	<u>62.16</u>	54.24 (54.21, 58.19)	53.05 (54.63, 56.89)	57.68 (57.90, 62.11)
	SupFam	50.56	67.52	67.24	72.30	61.97	<u>72.02</u>	55.91 (58.03, 66.31)	56.13 (58.25, 64.09)	61.08 (60.39, 69.20)
Average		51.90	65.37	68.30	69.38	69.24	72.43	61.19 (68.10, 73.57)	61.84 (68.30, <u>74.93</u>)	71.32 (71.65, 78.08)
Physicochemical Property Prediction, Spearman ρ (%) $\uparrow$										
FlexRMSF	Fold	15.35	13.81	44.22	33.15	<u>44.53</u>	44.63	18.05 (20.40, 24.37)	24.71 (29.08, 23.79)	37.12 (36.86, 33.12)
	SupFam	11.99	7.62	39.08	26.93	<u>39.68</u>	40.99	20.37 (21.21, 38.89)	25.67 (28.03, 30.52)	34.38 (34.84, 39.52)
FlexBFactor	Fold	4.17	6.67	22.32	18.88	<u>23.60</u>	21.30	13.66 (14.88, 18.87)	16.95 (15.79, 26.22)	21.66 (21.71, 22.44)
	SupFam	6.97	5.47	23.73	19.31	25.80	21.76	15.85 (15.40, 11.87)	17.52 (17.82, 20.38)	24.11 (24.50, 16.78)
FlexNEQ	Fold	5.71	12.98	35.95	16.41	<u>45.08</u>	49.64	15.41 (17.13, 17.37)	18.68 (21.43, 18.95)	29.48 (29.53, 26.18)
	SupFam	2.60	12.50	35.61	16.17	<u>45.43</u>	50.15	16.31 (18.33, 16.82)	21.51 (24.22, 23.48)	31.03 (30.79, 27.10)
Average		7.80	9.84	33.49	21.81	<u>37.35</u>	38.08	16.61 (17.89, 21.37)	20.84 (22.73, 23.89)	29.63 (29.71, 27.52)

Figure 10 shows the corresponding length distributions. PLLM moves toward the right-tailed UniRef50-like length regime by moderate temperatures, while direct amino acid generation keeps a large mass near the context-length cap even at $T = 1.0$. This supports treating the amino acid failure as a stopping-prior problem, not only a missing- $\langle\text{EOS}\rangle$ artifact. At $T = 1.0$, PLLM has mean length 277.1 residues, close to the UniRef50 mean of 273.6 residues. The amino acid model is much longer at the same temperature, with mean length 609.7 residues and 41.7% of samples near the context-length cap. Sampling hotter does not fix this tradeoff for amino acid generation. At $T = 2.0$, amino acid entropy improves, but the median length returns to the cap and 58.5% of samples are capped.

C.3.2. PLLM Conditional Generation

For the fixed-length sequence-side comparison, we use conditional generation for both the amino acid model and PLLM. The raw conditional diagnostics cover target lengths 100, 200, 300, and 500 and 10 temperatures from $T = 0.1$ to $T = 1.0$, with 200 generations per condition and 8 000 samples per model. The length token specifies the requested content length, but the model must still emit the $\langle\text{EOS}\rangle$ token to complete the sample. This gives a like-for-like comparison at the same intended lengths without truncating uncontrolled generations. We avoid post hoc length cutting because cutting a sequence before $\langle\text{EOS}\rangle$ can create incomplete samples and hide failures of the model’s stopping policy. Length conditioning nearly eliminates the target-length failure for both modalities.

Table 19: Full reconstruction comparison following GCP-VQVAE [38]. GCP-VQVAE Lite 2 denotes SLL. The grayed GCP-VQVAE Large column is not used for bold and underline ranking.

Dataset	Metric	GCP-VQVAE Lite 2	GCP-VQVAE Large [38]	GCP-VQVAE Lite [38]	AIDO [58]	ESM-3 VQVAE [16]	DPLM-2 VQVAE [49]	FoldToken 4 [12]	Gaujac et al. [14]	AminoAseed [54]
CASP14	TM-score $\uparrow$	0.9569	0.9890	<u>0.9751</u>	0.9831	0.9734	0.9056	0.8989	0.8940	0.3951
	RMSD $\downarrow$	1.1343	0.5431	<u>0.8435</u>	0.6587	1.0968	2.0896	2.0027	1.8913	14.6739
CASP15	TM-score $\uparrow$	0.9390	0.9884	0.9665	<u>0.9660</u>	0.9418	0.8268	0.9136	0.7911	0.3831
	RMSD $\downarrow$	2.1261	0.5293	<u>0.9219</u>	0.8611	1.3407	6.3826	1.6856	8.5592	18.5827
CASP16	TM-score $\uparrow$	<u>0.9542</u>	0.9857	0.9757	0.8805	0.9103	0.7852	0.8211	0.8345	0.3433
	RMSD $\downarrow$	<u>1.5030</u>	0.7567	1.0614	4.9773	4.4918	14.1934	5.4115	6.9375	25.9508
CAMEO-2024	TM-score $\uparrow$	0.9620	0.9918	<u>0.9794</u>	0.9851	0.9736	0.9243	0.9346	0.9114	0.4632
	RMSD $\downarrow$	1.0847	0.4377	<u>0.7401</u>	0.5867	1.0156	1.9410	1.4069	2.1726	12.6203
Zero-shot	TM-score $\uparrow$	0.9159	0.9673	0.9466	<u>0.9202</u>	0.8466	0.7137	0.8256	0.7662	0.4155
	RMSD $\downarrow$	<u>1.5131</u>	0.8193	1.1152	2.0105	6.5075	12.2182	4.5465	10.4405	19.2612
Average	TM-score $\uparrow$	0.9456	0.9844	0.9687	<u>0.9470</u>	0.9291	0.8311	0.8788	0.8394	0.4000
	RMSD $\downarrow$	<u>1.4722</u>	0.6172	0.9364	1.8189	2.8905	7.3650	3.0106	6.0002	18.2178

Table 20: Unsupervised PST full-run comparison following GCP-VQVAE [38]. GCP-VQVAE Lite 2 denotes SLL. The grayed GCP-VQVAE Large row is not used for bold and underline ranking.

Model	ApoHolo		FoldSwitching	
	PCC (%) $\uparrow$	Spearman (%) $\uparrow$	PCC (%) $\uparrow$	Spearman (%) $\uparrow$
ProteinMPNN [8]	35.80	45.22	51.46	55.91
MIF [51]	35.82	43.55	54.48	59.27
FoldSeek [46]	34.79	43.03	51.88	56.54
ProTokens [24]	43.32	54.05	61.20	65.90
ESM3 [16]	39.76	50.97	57.12	62.23
VanillaVQ [54]	39.62	50.61	56.51	61.51
AIDO [58]	43.02	54.25	61.59	66.11
AminoAseed [54]	42.47	54.88	65.61	75.89
GCP-VQVAE Large [38]	48.16	59.84	59.63	72.33
GCP-VQVAE Lite [38]	48.56	59.33	61.11	<u>73.94</u>
GCP-VQVAE Lite 2	<u>47.10</u>	<u>57.23</u>	60.69	67.95

The amino acid model reaches the requested length in 99.99% of samples, while PLLM reaches the exact requested length in 95.71% of samples and is within five residues in 96.98%. However, the entropy distribution remains substantially lower for amino acid generation, showing that its low-complexity drift is not only a length-control or <EOS> problem. Figures 11 to 13 provide the supporting conditional-generation diagnostics. The mean-entropy panel shows that PLLM approaches the UniRef50 entropy reference by moderate to high temperatures across target lengths, whereas amino acid generation remains lower. The length panel confirms that the length token largely controls the requested output size for both models. The full entropy distributions show that PLL concentrates near the natural high-entropy regime more consistently than amino acid generation.

Figure 14 shows the corresponding entropy-pLDDT correlation without removing drifted sequences. In this unfiltered view, the amino acid model keeps a stronger positive correlation across the temperature sweep, while PLL weakens toward zero and becomes negative at higher temperatures.

For the entropy-pLDDT correlation diagnostic in the right panel of Figure 6, we use target lengths 100,

200, 300, and 500 across temperatures $T = 0.1$ to $T = 1.0$. Each modality-length-temperature condition contributes 50 entropy-filtered ESMFold-folded sequences, giving 40 conditions and 2000 folded sequences per modality. For both modalities, we first retain non-drifted sequences with $H_{\text{seq}}(x) > 1.5$ bits, fold them with ESMFold [26], and parse mean pLDDT from the predicted structures. We then compare matched retained sets across amino acid and PLLM generation. When the direct amino acid model has fewer retained sequences than PLL, sampling is continued until the retained counts match. This prevents a trivial comparison in which low-complexity amino acid samples dominate the correlation estimate.

After this control, amino acid generation keeps a stronger positive entropy-pLDDT correlation, while PLL is near zero or negative around $T = 0.8$ to $T = 1.0$. We interpret this as pLDDT being more entangled with escape from low-complexity residue patterns in the amino acid model.

C.3.3. SLLM Length-Conditioned Continuation

After fully unconditional SLLM pretraining, we continue the largest SLLM for two epochs with a partial length-conditioning curriculum. The length token is prepended to 50% of training samples, while the remaining samples keep the unconditional format. Validation remains unconditional, so the loss change measures whether the curriculum also improves the base next-token model rather than only a matched conditional validation format.

Figure 15 shows that validation loss decreases from 3.891 at the epoch-eight boundary to 3.670 after the two continuation epochs, a 5.70% reduction. The same checkpoint is used for length-conditioned SLLM generation. After continuation, generated samples show nearly exact target-length control and are evaluated directly in the requested length buckets of 50, 100, 200, 300, 400, and 500 residues. This result suggests that length conditioning is more than a sampling convenience. It points to curriculum learning during pretraining as a useful way to improve the loss-compute frontier while enabling length-wise generation.

C.3.4. SLLM Sequence-to-Structure Model Selection

The sequence-to-structure checkpoint is initialized from the length-conditioned SLLM in Section C.3.3. It is fine-tuned with E1 sequence context and reaches the evaluation checkpoint after 64 000 optimizer steps and 4 315 655 500 unmasked next-token targets in the structure-token region. The model then generates SLL token streams with the `<sequence_to_structure>` condition and decodes selected streams with the fixed SLL decoder.

For confidence-based selection, we tune the DeepConf-style selector on the combined CASP14, CASP15, and CASP16 set. This tuning set contains 118 targets and uses 64 generated candidates per target. CAMEO-2024 is reserved for verification after the selector is fixed, with 574 targets and 256 generated candidates per target. Both runs use temperature 1.0 and top- p sampling with threshold 0.90.

The selected configuration scores each candidate by the maximum mean raw full-vocabulary entropy over sliding windows of 256 generated SLL tokens, using stride one. Lower score means higher internal confidence. We retain the most confident 50% of candidates, require at least two retained samples, and select the token-similarity medoid among the retained set. This is the offline DeepConf-style rule described in Section A.6.

Figures 16 and 17 show the full inference-time scaling curves. Generation is fast enough to make this selection regime practical. With KV caching, FA2, and model compilation, the sequence-to-structure model reaches about 1 400 generated tokens per second on two NVIDIA RTX 6000 Ada GPUs, or about three completed samples per second on this workload. Longer targets mainly add more generated tokens, so

cached autoregressive decoding scales close to linearly with sequence length. The fixed SLL decoder adds negligible overhead after token generation. In our measurements, candidate-token generation for long proteins is approximately 1 000 times faster than MSA-based AlphaFold2 [22] running on an NVIDIA A100 GPU.

The oracle best@ k curves improve approximately log-linearly with the number of sampled candidates. This is not an inference method because it uses the native structure, but it shows that the sampled SLL candidate pool contains better structures as k increases. On the CASP14/15/16 tuning set, DeepConf selection at $k = 64$ improves average TM-score from 0.564 for a single sample to 0.589, while oracle best@64 reaches 0.667. On CAMEO-2024, the fixed selector at $k = 256$ improves average TM-score from 0.725 to 0.754, while oracle best@256 reaches 0.825. The gap between DeepConf and oracle best@ k is therefore useful headroom, not a failure of inference-time scaling. It indicates that fast generation can partially compensate for the lower single-sample quality of the model, and that internal confidence from latent-token prediction can recover part of this gain without an external structure-quality metric.

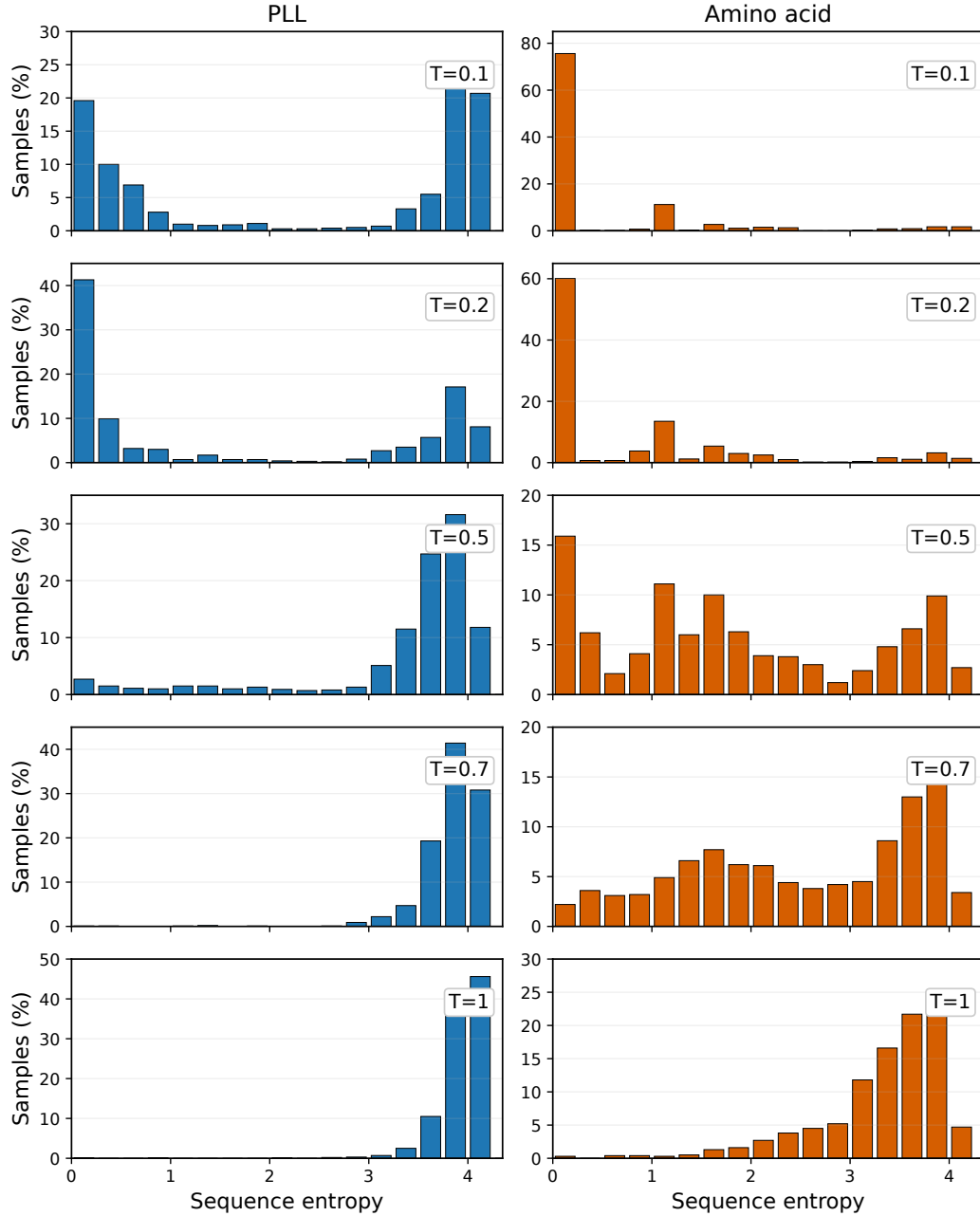


Figure 9: Unconditional sequence-entropy distributions for PLLM and amino acid generation across sampling temperatures. PLLM approaches the UniRef50-like high-entropy regime around $T = 0.6$ to $T = 1.0$.

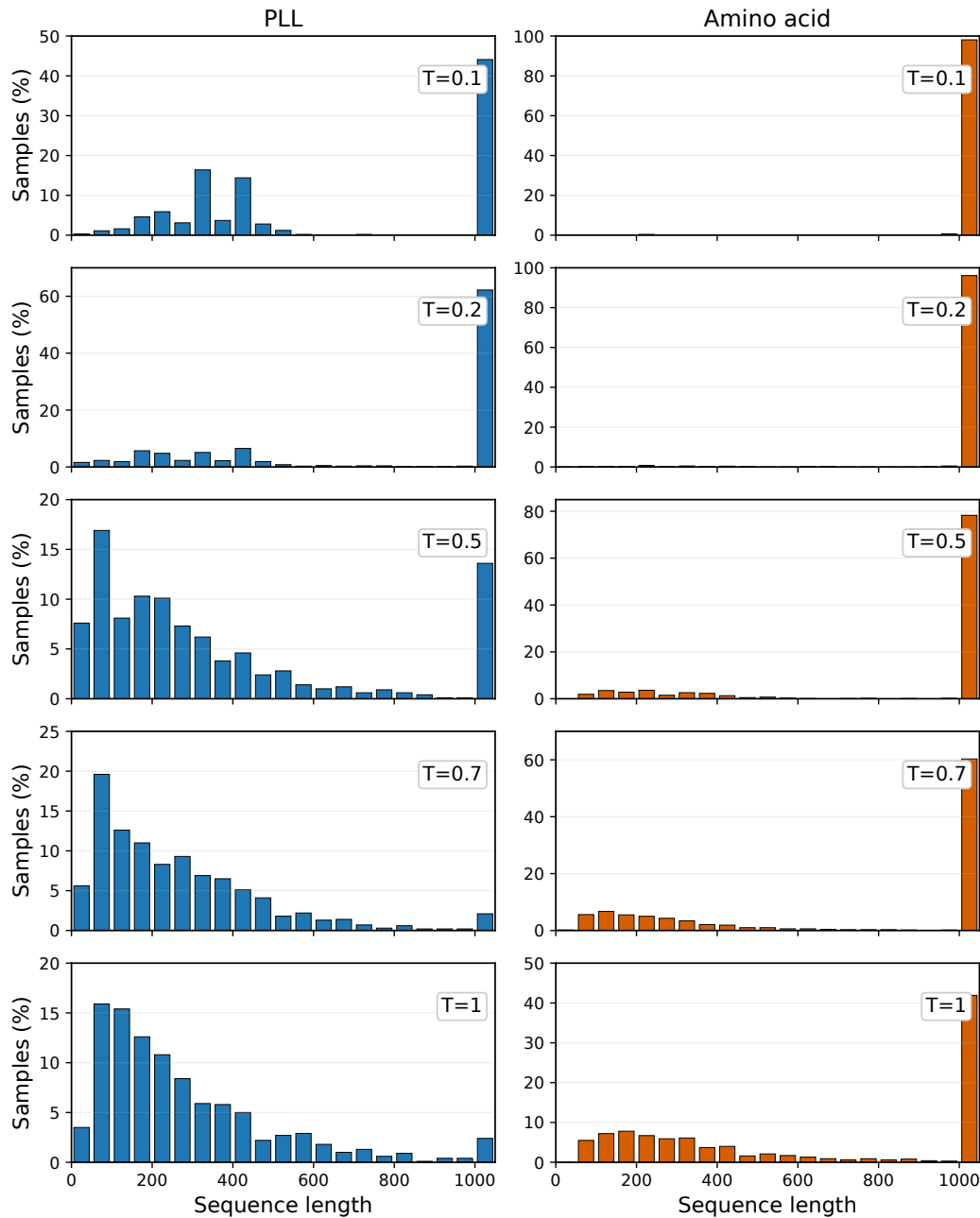


Figure 10: Unconditional length distributions for PLLM and amino acid generation across sampling temperatures. Direct amino acid generation overproduces capped sequences, while PLLM is closest to the UniRef50 length regime around $T = 0.6$ to $T = 1.0$.

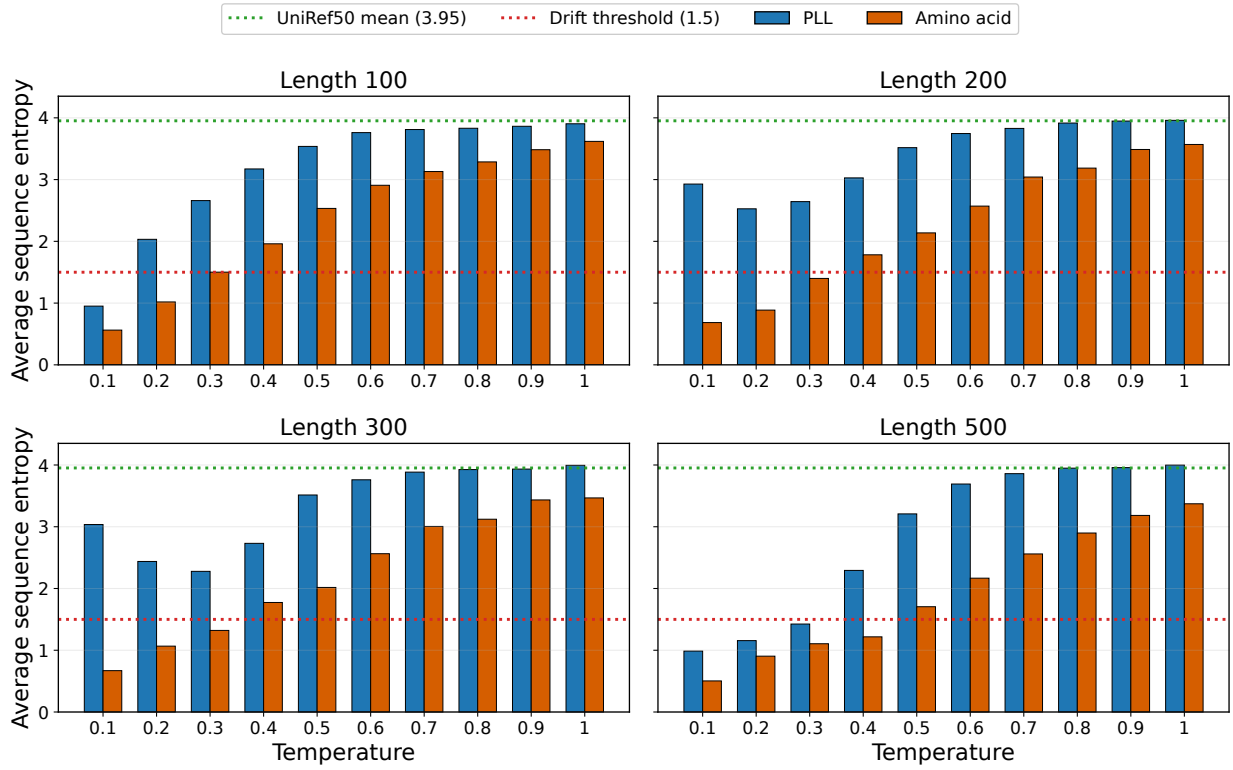


Figure 11: Length-conditioned mean sequence entropy across temperatures and target lengths. The green line marks the UniRef50 mean entropy, and the red line marks the drift threshold.

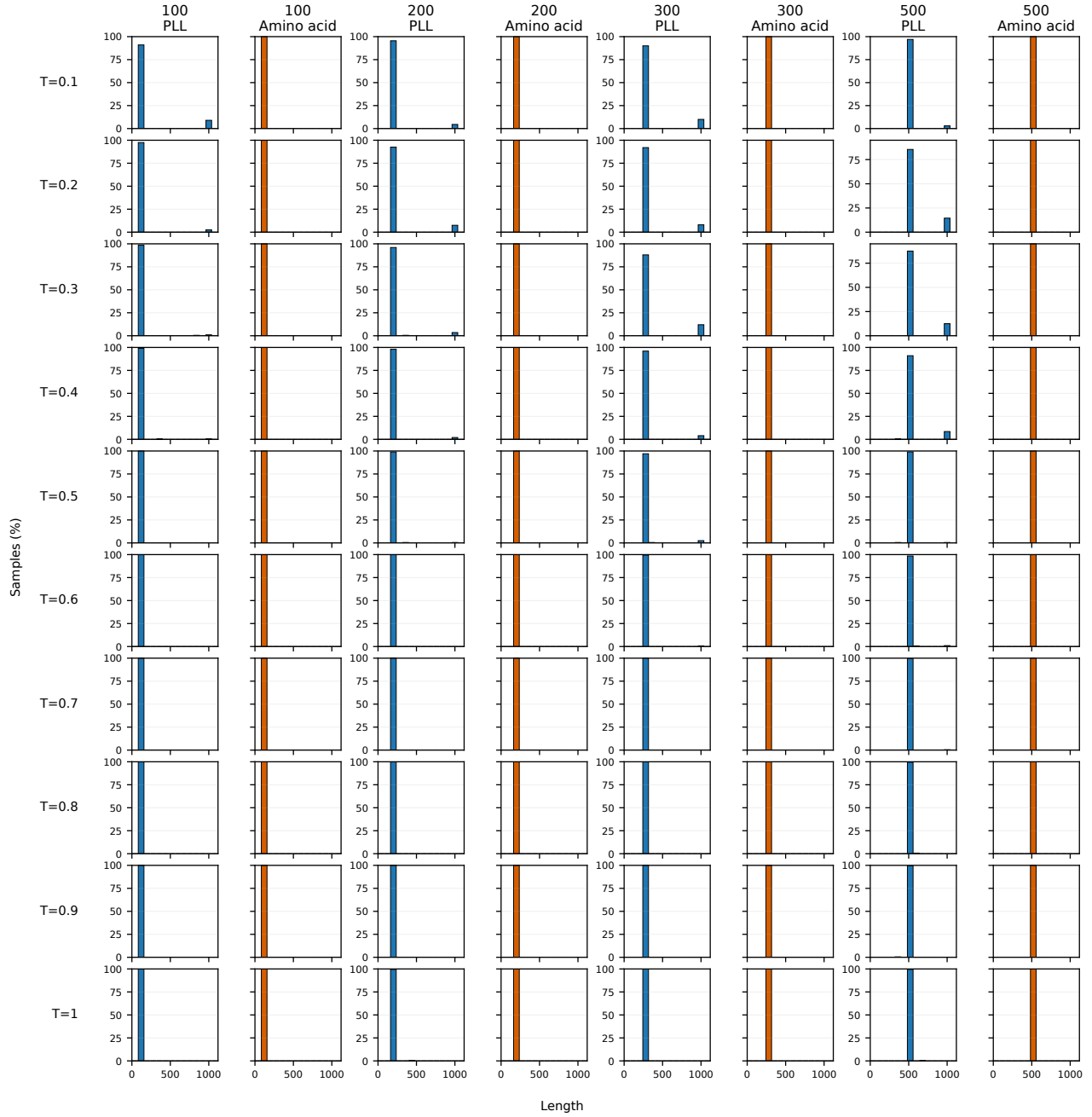


Figure 12: Length-conditioned sequence-length distributions across target lengths and temperatures. Both modalities concentrate near the requested length, showing that the length prefix largely fixes the stopping and target-size issue.

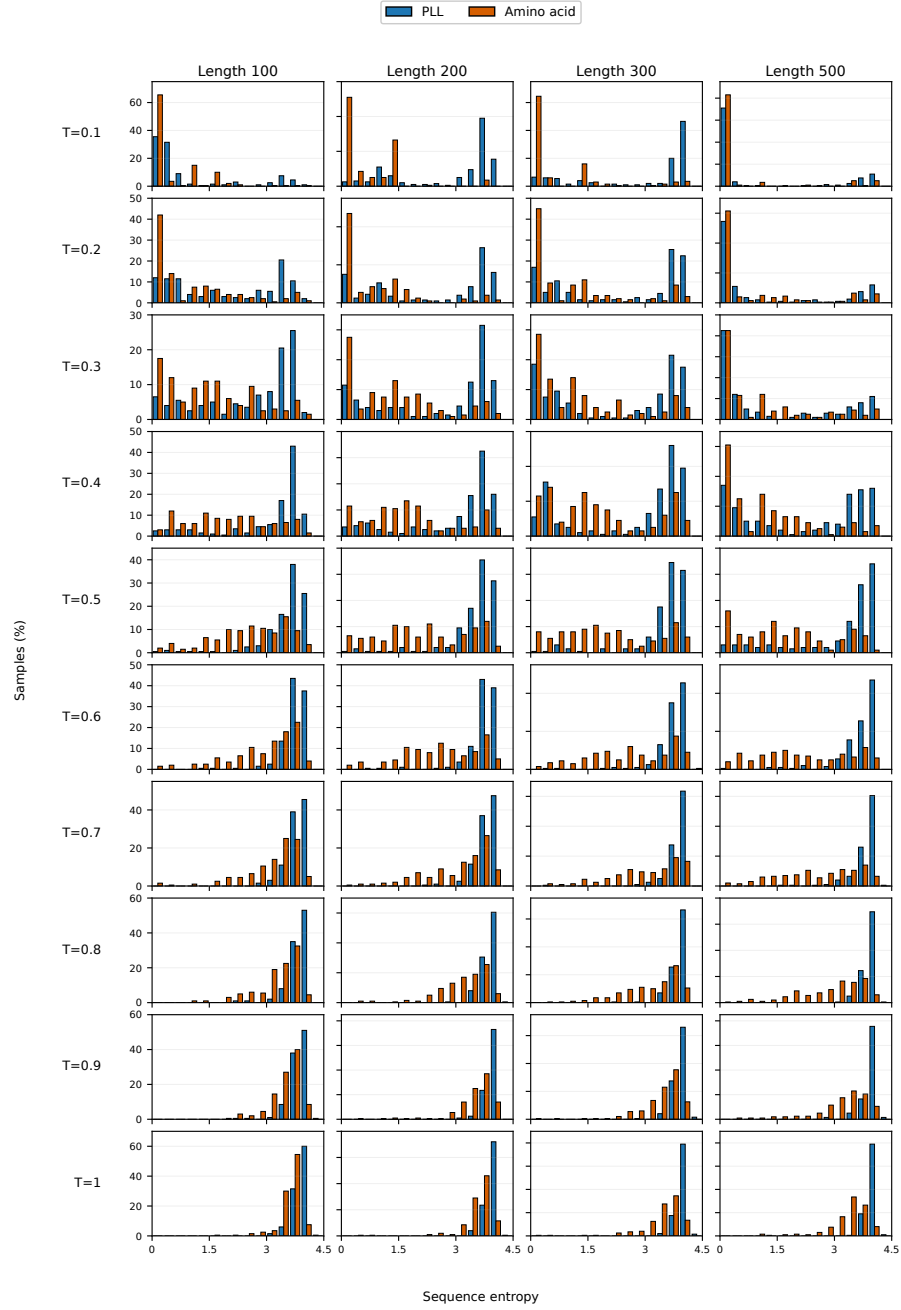


Figure 13: Length-conditioned sequence-entropy distributions across target lengths and temperatures. PLL shifts into the high-entropy natural-protein regime more consistently than amino acid generation.

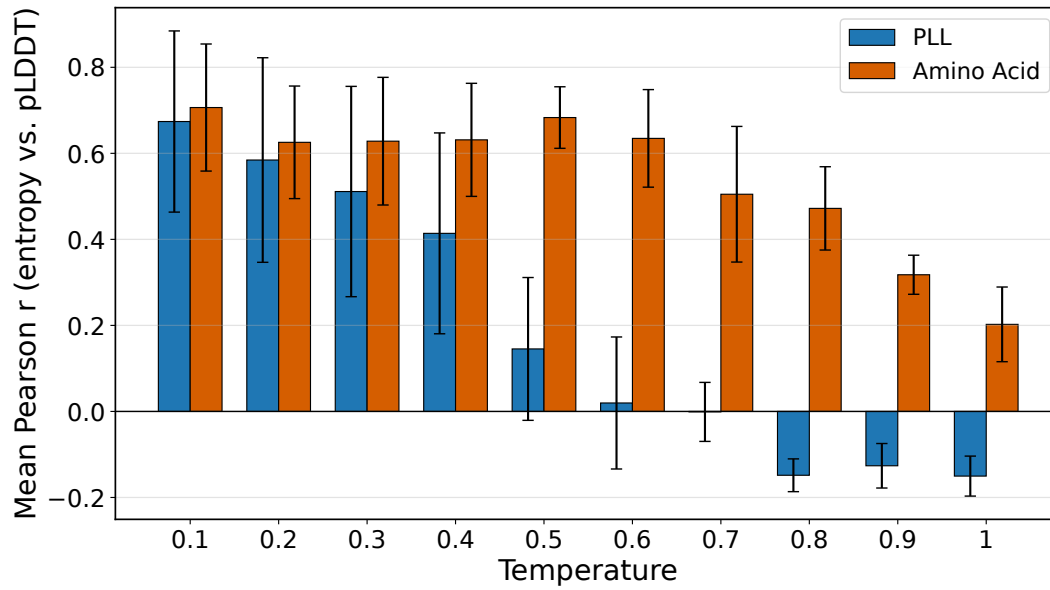


Figure 14: Length-conditioned entropy-pLDDT correlation without removing drifted sequences.

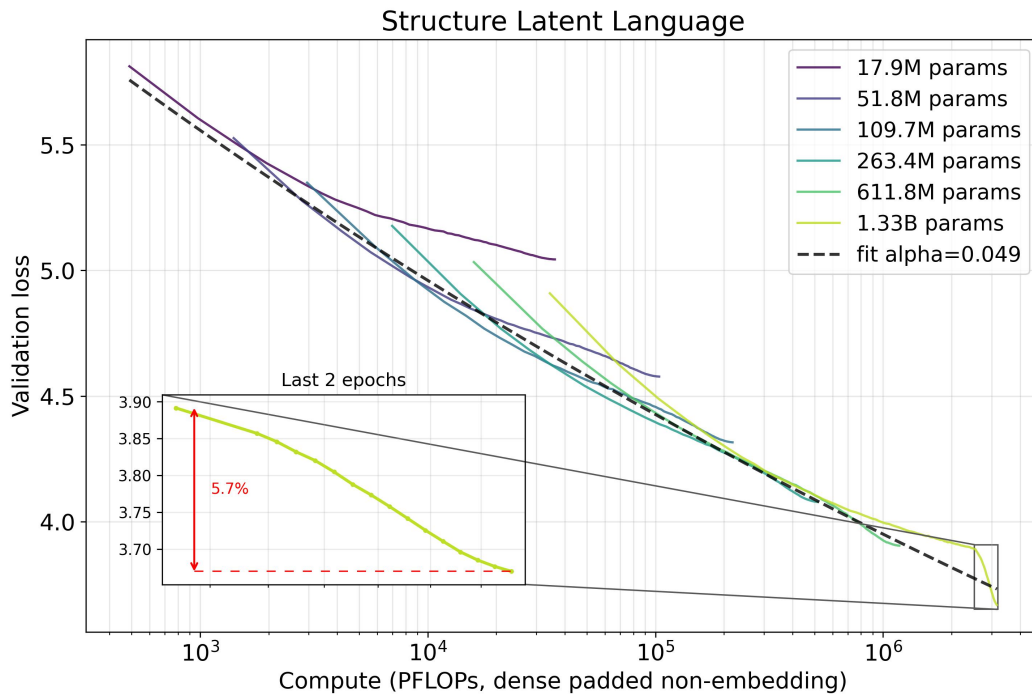


Figure 15: SLL length-conditioned continuation after unconditional pretraining. A two-epoch curriculum with length tokens on 50% of samples lowers unconditional validation loss and adds a length-control interface for generation.

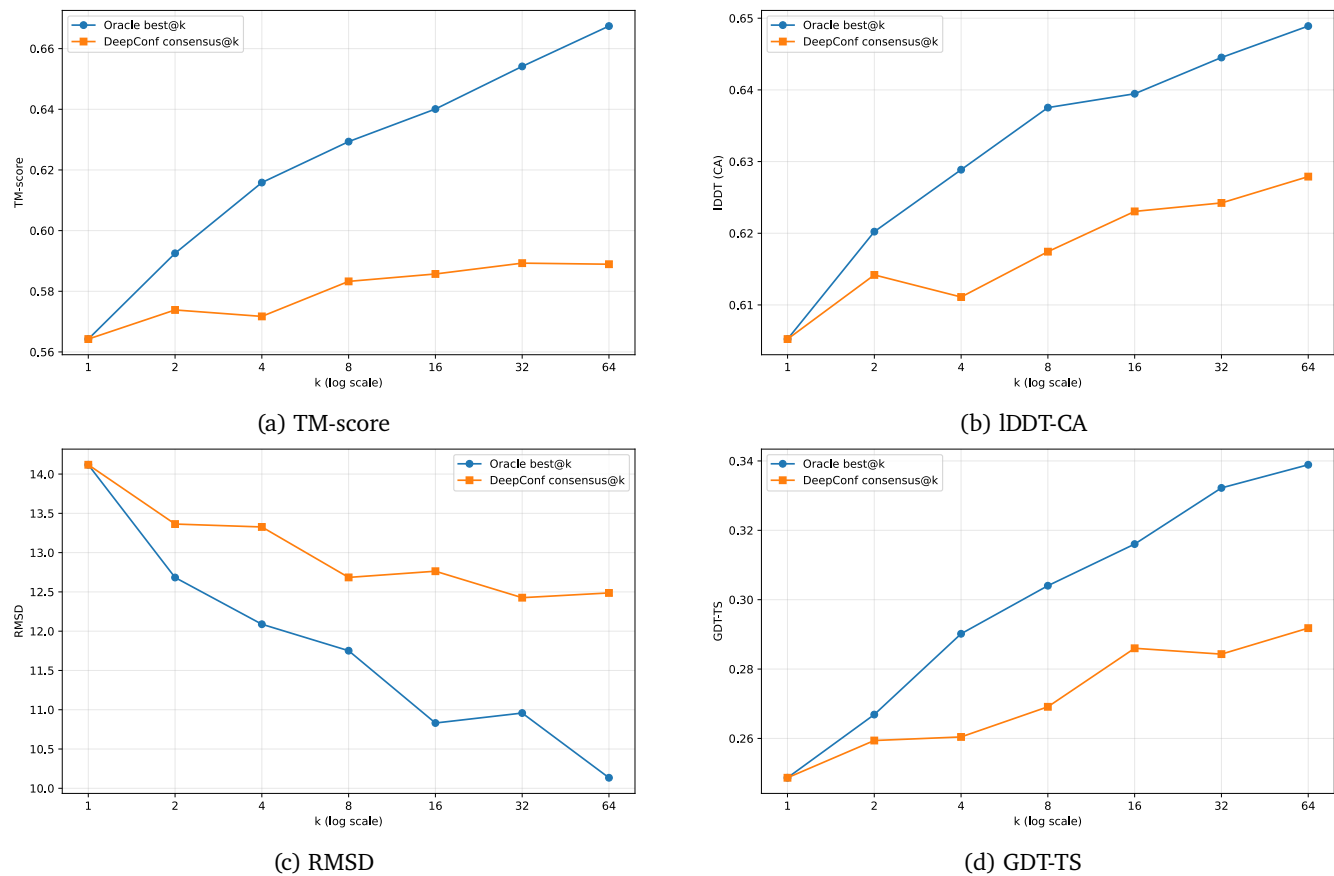


Figure 16: Sequence-to-structure inference-time scaling on the combined CASP14, CASP15, and CASP16 tuning set. DeepConf uses internal confidence without ground truth, while oracle best@k is a ceiling on the sampled candidate pool.

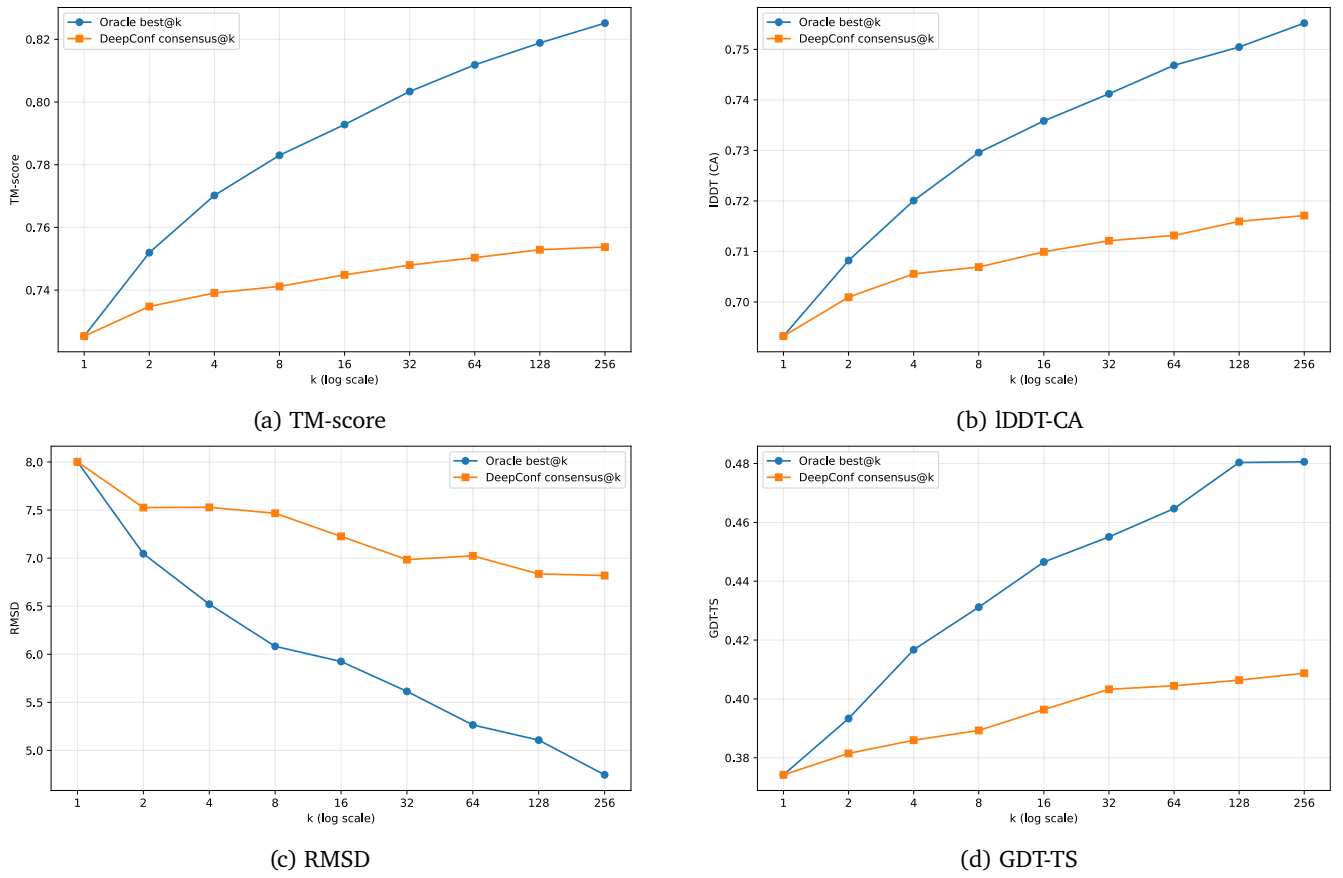


Figure 17: Sequence-to-structure inference-time scaling on CAMEO-2024 after fixing the selector on CASP14/15/16.